

03. Grids and BFS/DFS

This is the chapter everything else in robotics path planning is built on. A robot's map is a grid. "Find the shortest way to the goal" is BFS. "Which cells can the robot reach" is DFS. Almost every grid question you will be asked is one of four patterns, and this chapter teaches each pattern from zero, with a hand trace, the Python you would have written, and then the C++.

Rough time: 6 days at about 2 hours a day. Day 1 is Part A only. Do not start the problems until you can explain BFS and DFS in your own words from Part A.

What you need before this chapter (all in chapter 01): `std::vector<std::vector<int>>`, `std::pair`, `std::queue`, `auto [a, b] = ...`, and how to compile with `g++`. If any of those look foreign, go back to chapter 01 first; this chapter will not make sense otherwise.

Part A — The four patterns, from zero

A1. What a grid is, and how to index it

A grid is a table of cells. Here is a 3-row by 4-column grid with a wall in it (# is a wall, . is free):

	c=0	c=1	c=2	c=3
r=0	.	.	.	.
r=1	.	#	#	.
r=2	.	.	.	.

Two facts you must have burned in:

1. **grid[r][c] means row r, then column c.** *r* grows *downwards*, *c* grows to the *right*. So `grid[1][2]` is row 1, column 2: the second #. This is the same as Python's `grid[r][c]` on a list of lists. It is *not* (x, y); if you think "x then y" you will swap them and get a bug that is very hard to see.
2. **The size.** `rows = grid.size()` and `cols = grid[0].size()`. In Python that was `len(grid)` and `len(grid[0])`.

In Python you would write the grid above as:

```
grid = [
    ['.','.','.','.'],
    ['.','#','#','.'],
    ['.','.','.','.'],
]
rows, cols = len(grid), len(grid[0])
```

In C++ it is the same thing with types written out:

```
std::vector<std::vector<char>> grid = {
    {'.','.','.','.'},
    {'.','#','#','.'},
    {'.','.','.','.'},
};
int rows = static_cast<int>(grid.size());
int cols = static_cast<int>(grid[0].size());
```

(You can skip this paragraph on a first read.) Why the `static_cast<int>?` `size()` gives back an "unsigned" number, a type that cannot be negative. Mixing it with ordinary `ints` makes the compiler warn. Converting it to `int` once at the top, and using `int` everywhere after, keeps things simple. Just copy those two lines in every grid problem.

Neighbours. A cell's 4 neighbours are up, right, down, left. For the cell at (r, c) :

	$(r-1, c)$	up	
$(r, c-1)$	(r, c)	$(r, c+1)$	left, me, right
	$(r+1, c)$	down	

Instead of writing four separate `ifs`, everyone stores the four *offsets* in two little arrays and loops over them. Python:

```
for dr, dc in [(-1, 0), (0, 1), (1, 0), (0, -1)]: # up, right, down, left
    nr, nc = r + dr, c + dc
```

C++:

```
const int dr[4] = {-1, 0, 1, 0}; // change in row: up, right, down, left
const int dc[4] = {0, 1, 0, -1}; // change in column: up, right, down, left
for (int k = 0; k < 4; ++k) {
    int nr = r + dr[k];
    int nc = c + dc[k];
    // ... use (nr, nc)
}
```

Read it as: "for each of the 4 directions k , the neighbour is my row plus $dr[k]$, my column plus $dc[k]$ ". nr/nc mean "new row / new column". You will type these two arrays in every grid problem for the rest of your life.

Staying inside the grid. The cell at $(0, 0)$ has no "up" neighbour: $(-1, 0)$ is outside. In Python, `grid[-1][0]` silently gives you the *last* row, which is a bug you never notice. In C++, `grid[-1][0]` is undefined behaviour: it may crash, or may silently read garbage. So every grid algorithm has this check, and it must come *before* you touch `grid[nr][nc]`:

```
bool inBounds(int r, int c, int rows, int cols) {
    return r >= 0 && r < rows && c >= 0 && c < cols;
}
// ...
if (!inBounds(nr, nc, rows, cols)) continue; // skip neighbours outside the grid
if (grid[nr][nc] == '#') continue; // skip walls; only safe AFTER the bounds check
```

That is all the grid machinery there is: `grid[r][c]`, the `dr/dc` arrays, and `inBounds`. Everything below is built from those three pieces.

In your own words: "I index the grid as row then column, keep the four direction offsets in two arrays, and always check bounds before I read a neighbour."

A1b. Three tools you need: a queue, a stack, and recursion

Everything in this chapter is built from three simple tools. If you know them, skip ahead.

A queue is a line at a shop. People join at the back and are served from the front, so whoever arrived first is served first. In C++:

```
#include <queue>
std::queue<int> q;
q.push(5); // 5 joins the back of the line
q.push(7); // 7 joins behind it
int x = q.front(); // look at who is first: 5
q.pop(); // the first person leaves (this returns nothing; you looked with front() already)
bool e = q.empty(); // is the line empty? no, 7 is still there
```

Python's `collections.deque` with `append` and `popleft` is the same thing. The one habit to change: in C++, "look" (`front()`) and "remove" (`pop()`) are two separate steps.

A stack is a pile of plates. You put a plate on top, and you take a plate from the top, so the *last* one put on is the *first* one taken off. In C++:

```
#include <stack>
std::stack<int> st;
st.push(5);      // plate 5 on the pile
st.push(7);      // plate 7 on top of it
int x = st.top(); // look at the top plate: 7
st.pop();        // take it off; 5 is on top now
```

A Python list with `append` and `pop()` is a stack.

Recursion is a function that calls itself on a smaller version of the problem. Think of Russian dolls: to open the whole set, open one doll and then "open the whole set" that is inside, which is the same task but smaller, until you reach the tiny solid doll and stop. The "stop" rule is called the base case, and every recursive function must have one or it runs forever.

```
void countDown(int n) {
    if (n == 0) return; // base case: nothing left to do
    std::cout << n << "\n";
    countDown(n - 1);   // the same task, one smaller
}
```

Each call has its own copy of `n`. The computer keeps a pile of the unfinished calls (that pile is literally called the call stack), and when a call finishes, the one below it carries on. That is what lets DFS "back up" for free.

A2. BFS: the ripple

The picture. Drop a stone into a pond at the start cell. The ripple spreads outward one ring at a time: first all cells 1 step away, then all cells 2 steps away, and so on. When the ripple first touches the goal, the ring number *is* the shortest distance. That is Breadth-First Search, and that is why it finds shortest paths: it explores in order of distance.

The tool. A *queue*: you add cells to the back and take cells from the front, so cells are processed in the order they were discovered. Cells discovered at distance 1 are all processed before any cell discovered at distance 2.

The hand trace. Start at `S = (0, 0)` in the grid from A1. We keep a `dist` table, all cells set to `-1` meaning "not reached yet". Reaching a cell means writing its distance and putting it in the queue.

Step 0: `dist[0][0] = 0`, `queue = [(0,0)]`.

```
dist:  0  -1  -1  -1
      -1  #   #  -1
      -1  -1  -1  -1
```

Step 1: take `(0,0)` from the front. Its neighbours: up is out of bounds, right is `(0,1)`, down is `(1,0)`, left is out of bounds. Both unreached, so they get distance `0 + 1 = 1` and go in the queue. `queue = [(0,1), (1,0)]`.

```
dist:  0  1  -1  -1
      1  #   #  -1
     -1  -1  -1  -1
```

Step 2: take `(0,1)`. Neighbours: right `(0,2)` unreached → dist 2. Down `(1,1)` is a wall, skip. Left `(0,0)` already has a distance, skip. `queue = [(1,0), (0,2)]`.

Step 3: take (1,0). Down (2,0) unreachable → dist 2. queue = [(0,2), (2,0)].

```
dist:    0    1    2    -1
         1    #    #    -1
         2   -1   -1   -1
```

Step 4: take (0,2). Right (0,3) → dist 3. queue = [(2,0), (0,3)]. Step 5: take (2,0). Right (2,1) → dist 3. queue = [(0,3), (2,1)].

```
dist:    0    1    2    3
         1    #    #   -1
         2    3   -1   -1
```

Keep going and the table fills in:

```
dist:    0    1    2    3
         1    #    #    4
         2    3    4    5
```

Notice: every cell was reached exactly once, and its number is the true shortest distance from S. The queue processed all the 1s before the 2s, all the 2s before the 3s. That ordering is the whole reason BFS works.

The Python you would have written.

```
from collections import deque

def bfs(grid, sr, sc):
    rows, cols = len(grid), len(grid[0])
    dist = [[-1] * cols for _ in range(rows)]
    dist[sr][sc] = 0
    q = deque([(sr, sc)])
    while q:
        r, c = q.popleft()
        for dr, dc in [(-1, 0), (0, 1), (1, 0), (0, -1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] != '#' and dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))
    return dist
```

The same thing in C++. Read the two side by side; every line has a twin.

```
#include <queue>
#include <utility>    // std::pair
#include <vector>

std::vector<std::vector<int>> bfs(const std::vector<std::vector<char>>& grid, int sr, int sc) {
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    std::vector<std::vector<int>> dist(rows, std::vector<int>(cols, -1));    // rows x cols, all -1
    const int dr[4] = {-1, 0, 1, 0};
    const int dc[4] = {0, 1, 0, -1};

    std::queue<std::pair<int, int>> q;    // a queue of (row, col) pairs
    dist[sr][sc] = 0;
    q.push({sr, sc});

    while (!q.empty()) {
        auto [r, c] = q.front();    // look at the front...
        q.pop();                    // ...then remove it. Two calls, unlike popleft().
        for (int k = 0; k < 4; ++k) {
            const int nr = r + dr[k];
            const int nc = c + dc[k];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;    // outside the grid
```

```

        if (grid[nr][nc] == '#') continue;           // wall
        if (dist[nr][nc] != -1) continue;           // already reached
        dist[nr][nc] = dist[r][c] + 1;
        q.push({nr, nc});
    }
}
return dist;
}

```

Python	C++	Note
<code>deque([(sr, sc)])</code>	<code>std::queue<std::pair<int,int>> q; q.push({sr, sc});</code>	You must say what the queue holds: pairs of ints.
<code>r, c = q.popleft()</code>	<code>auto [r, c] = q.front(); q.pop();</code>	C++ <code>pop()</code> returns nothing. Look with <code>front()</code> , then <code>pop()</code> .
<code>[[[-1]*cols for _ in range(rows)]]</code>	<code>std::vector<std::vector<int>> dist(rows, std::vector<int>(cols, -1));</code>	"rows copies of a row of cols minus-ones".
<code>0 <= nr < rows</code>	<code>nr >= 0 && nr < rows</code>	No chained comparisons in C++.
<code>q.append((nr, nc))</code>	<code>q.push({nr, nc});</code>	<code>{nr, nc}</code> builds the pair.
<code>while q:</code>	<code>while (!q.empty())</code>	A queue is not "truthy" in C++.

The one rule that people get wrong. Mark a cell as reached (write its `dist`) *when you push it*, not when you pop it. If you wait until popping, the same cell can be pushed by several neighbours before it is popped, the queue fills with duplicates, and the distances are still right but level counting (next paragraph) breaks. Marking on push means every cell enters the queue exactly once.

Variant 1: counting rings ("how many minutes / steps"). Sometimes you do not want a distance per cell, you want "how many rounds until everything is reached". Process the queue one ring at a time: look at how many cells are in the queue right now, pop exactly that many, and then count one round.

```

int rounds = 0;
while (!q.empty()) {
    const int ringSize = static_cast<int>(q.size()); // how many cells are in THIS ring
    for (int i = 0; i < ringSize; ++i) {
        auto [r, c] = q.front(); q.pop();
        // ... push unreached neighbours as before ...
    }
    ++rounds; // one whole ring done
}

```

The `ringSize` snapshot matters: the queue grows while you are inside the loop, so you must record its size before you start.

Variant 2: many starting points ("multi-source BFS"). If there are several starts, push *all* of them with distance 0 before the loop. Nothing else changes. Each cell then ends up with the distance to the *nearest* start. Rotting Oranges and 01 Matrix are exactly this.

In your own words: "BFS uses a queue so cells are processed in order of distance. I mark a cell when I push it so it enters the queue once. The first time I reach a cell is by a shortest path. For several starting points I push all of them first; for a round count I process the queue one ring at a time."

A3. DFS: walk until stuck, then back up

The picture. You are in a maze with a piece of chalk. Walk forward, marking every cell you step on. When you cannot go any further, back up to the last cell that still has an unmarked neighbour and try that. Eventually every cell connected to your start is marked. That is Depth-First Search. It does *not* find shortest paths (you may wander a long way round), but it answers "what can I reach from here?" and "how many separate regions are there?", which is what most DFS problems ask.

The tool. Recursion. "Visit this cell, then visit each unvisited neighbour" is a function that calls itself. The call stack does the "back up" for you: when the recursive call returns, you are back at the previous cell.

The hand trace. Same grid, start at (0,0), mark with x:

```
visit (0,0) -> mark. Try up: out. Try right (0,1):
  visit (0,1) -> mark. Try up: out. Try right (0,2):
    visit (0,2) -> mark. Try right (0,3):
      visit (0,3) -> mark. up out, right out, down (1,3):
        visit (1,3) -> mark. down (2,3):
          visit (2,3) -> mark. down out, left (2,2):
            visit (2,2) -> mark. ... this keeps going along the bottom row ...
            ... (2,1), (2,0), (1,0) all marked, then it is stuck: everything around is marked or a wall
            returns, returns, returns ... all the way back up to (0,0)
(0,0) tries down (1,0): already marked. Done.
```

Every free cell got marked exactly once. The path taken was long and wiggly, and that is fine for "can I reach it", not for "how far is it".

The Python you would have written.

```
def dfs(grid, r, c, visited):
    rows, cols = len(grid), len(grid[0])
    if not (0 <= r < rows and 0 <= c < cols):    # outside
        return
    if grid[r][c] == '#' or visited[r][c]:        # wall or already marked
        return
    visited[r][c] = True
    for dr, dc in [(-1, 0), (0, 1), (1, 0), (0, -1)]:
        dfs(grid, r + dr, c + dc, visited)
```

Notice the shape: the checks that *stop* the recursion are at the top, then mark, then recurse on the four neighbours. That shape is every DFS.

The same thing in C++.

```
void dfs(const std::vector<std::vector<char>>& grid, int r, int c,
         std::vector<std::vector<bool>>& visited) {
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    if (r < 0 || r >= rows || c < 0 || c >= cols) return;    // outside
    if (grid[r][c] == '#' || visited[r][c]) return;          // wall or already marked
    visited[r][c] = true;
    dfs(grid, r - 1, c, visited);    // up
    dfs(grid, r, c + 1, visited);    // right
    dfs(grid, r + 1, c, visited);    // down
    dfs(grid, r, c - 1, visited);    // left
}
```

`visited` is passed by reference (&) so all the recursive calls share one table. Without the &, each call would get its own copy and nothing would ever count as visited. This is the single most common C++ mistake in this chapter.

A cheaper "visited". Very often you are allowed to change the grid itself. Then instead of a separate visited table you overwrite the cell: turn '1' into '0', turn the fresh orange into a rotten one, turn the old colour into the new colour. The overwrite *is* the mark. It is less code and less memory; just be aware you are modifying the input, and copy it first if the caller still needs the original.

When recursion is too deep. Each recursive call uses a little stack memory, and the stack is about 8 MB on Linux. On a 1000 by 1000 grid that is all free cells, DFS can recurse a million levels deep and crash. Python would raise `RecursionError`; C++ just dies with a segmentation fault. The fix is to run DFS with an explicit stack instead of recursion: it is the BFS loop from A2 with `std::stack` in place of `std::queue` and `top()` in place of `front()`. You will see it in Number of Islands. Use recursion while learning; switch to the explicit stack when the grid can be large.

BFS or DFS?

You want...	Use	Why
Shortest distance / fewest steps	BFS	Explores in order of distance
"Can I reach it" / "which cells are connected"	Either; DFS is shorter to write	Both visit everything reachable
Count separate regions (islands)	DFS (or BFS) from each unvisited cell	Each start that finds new cells is a new region
Try every path and undo (Word Search)	DFS with backtracking (A4)	Needs the "back up" behaviour
Nearest-of-many (distance to closest 0)	Multi-source BFS	Push all sources first

In your own words: "DFS visits a cell, marks it, and recurses into unvisited neighbours; the call stack backs up for me. It finds everything reachable but not shortest paths. On a large grid recursion can overflow the stack, so I would use an explicit stack instead."

A4. Backtracking: mark, try, unmark

Plain DFS marks a cell once and leaves it marked, because it only cares *whether* a cell can be reached. Some problems ask for a *specific path*: "does the word CAT appear as a path of adjacent cells, using each cell at most once?" Now a cell being used on one attempted path must become free again when that attempt fails, so a different path can use it.

The pattern is three lines around the recursion:

```
mark the cell as in use
try each neighbour (recurse)
unmark the cell          <-- this is the only difference from DFS
```

Tiny trace. Looking for "AB" in

```
A  B
B  A
```

Start at $(0,0) = A$, matches letter 0. Mark it. Look for B next to it: $(0,1)$ is B, matches letter 1, that was the last letter, return true. Done. Now suppose the word were "ABA": from $(0,1)$ we need an A: $(1,1)$ is A, found. But if $(1,1)$ had been B, the attempt fails, we *unmark* $(0,1)$, back at $(0,0)$ we try down $(1,0) = B$, and so on. The unmark is what lets $(0,1)$ be reused by a later attempt starting somewhere else.

The cost: DFS visits each cell once, so it is linear in the grid size. Backtracking can revisit cells on different paths, so its worst case is exponential in the path length. That is expected for "find a specific path" problems and there is no way round it.

In your own words: "Backtracking is DFS where the visited mark is temporary: mark, recurse, unmark, so each cell can be reused on a different path. It is exponential in the worst case, which is unavoidable for search-for-a-path problems."

A5. DP on a grid: fill in a table

Some grid problems are not about searching at all. "How many ways can a robot walk from the top-left to the bottom-right if it can only move right or down?" There is no wandering here; each cell's answer depends only on the cell above and the cell to its left.

The picture. Fill in a table. The number of ways to reach a cell is (ways to reach the cell above) + (ways to reach the cell to the left), because the last step came from one of those two. The top row and left column are all 1 (only one way: straight along the edge).

ways:	1	1	1	1
	1	2	3	4
	1	3	6	10

`ways[2][3] = ways[1][3] + ways[2][2] = 4 + 6 = 10.`

That is dynamic programming on a grid: define what each cell means (`ways[r][c]` = number of paths to `(r, c)`), write how it comes from its neighbours (the *transition*), fill in the edges (*base cases*), and loop. With obstacles, an obstacle cell gets 0. For "minimum cost path", replace the sum with `cost[r][c] + min(above, left)`.

In your own words: "Grid DP: I define what each cell stores, state the transition from the cell above and the cell to the left, set the base cases along the edges, then fill the table row by row. Time is rows times columns."

A6. Why this matters for robots

A mobile robot's map is an *occupancy grid*: a 2D array of cells, each free, occupied, or unknown. In ROS this is `nav_msgs/OccupancyGrid`, a flat array of bytes with a width and height, exactly the `r * cols + c` flat indexing you will see in some solutions.

- **BFS from the goal** across the free cells gives every cell its distance to the goal. That is the *wavefront planner*: the robot just walks downhill. Rotting Oranges is a wavefront.
- **Multi-source BFS from every obstacle** gives every cell its distance to the nearest obstacle. That is the *distance transform*, and it is how a costmap's *inflation layer* is built (cells near walls get a high cost so the planner keeps its distance). 01 Matrix is exactly this.
- **DFS / flood fill from the robot** gives the set of reachable cells, used to find frontiers for exploration and to reject unreachable goals.
- **A*** is BFS's grown-up sibling for weighted grids, and it is what the global planner in the ROS navigation stack actually runs. You build it at the end of this chapter.

So every grid problem below is really the inner loop of a planner. When you see one, name the robotics version in your head: "this is a wavefront", "this is the distance transform".

A7. A* in one paragraph (details in Part C)

BFS treats every step as cost 1 and explores rings blindly in all directions. A *does two things differently*. First, it allows steps to have different costs (a diagonal costs about 1.414, a muddy cell costs more), so instead of a plain queue it uses a priority queue that always pops the cheapest cell so far. That alone is called Dijkstra. Second, it adds a guess of the remaining distance to the goal (for example the straight-line distance) to each

cell's priority, so the search leans toward the goal instead of spreading in a circle. If the guess never over-estimates, the first time the goal is popped the path is optimal. You will write this from scratch, with tests, in Part C. Understand BFS thoroughly first; A is BFS with a smarter queue.

A8. The toolkit file

code/03-grids-bfs-dfs/00_grid_toolkit.cpp contains the BFS and DFS functions from A2 and A3 (single-source BFS, multi-source BFS, recursive DFS, stack-based DFS) on a small map, with asserts. Compile and run it, then change the map and predict the output before you run it again:

```
cd code
g++ -std=c++17 -Wall -Wextra -O2 03-grids-bfs-dfs/00_grid_toolkit.cpp -o toolkit && ./toolkit
```

You do not need to memorise that file. You need to be able to type A2's BFS and A3's DFS from a blank screen. Practise that now, before the problems.

Part B — The problems

Each problem follows the same shape: the full problem in plain words with a drawn example, how you would solve it by hand, the Python you would have written, the C++ solution line by line, a walk-through of the example, and the follow-up questions that usually come with it.

The rule for using this part: read the problem statement, close the file, and try it in C++ for 25 minutes. Then read the rest.

1. Robot Return to Origin (LeetCode 657, easy)

The problem in plain words. A robot starts at the point (0, 0). You are given a string of moves, each character one of U (up), D (down), L (left), R (right). Each move is one step. Return true if, after all the moves, the robot is back at (0, 0), otherwise false.

```
moves = "UD"      -> up one, down one      -> back at origin -> true
moves = "LL"      -> left twice, now at (-2, 0) -> not at origin -> false
moves = "RRDD"    -> ends at (2, -2)        -> false
```

The string can be empty (then the robot never moved, answer true), and it can be up to 20,000 characters.

By hand. You do not need to draw the path. Count how far left versus right, and how far up versus down. The robot is back home only if the counts cancel: as many L as R, and as many U as D.

The idea. Keep two counters x and y. R adds 1 to x, L subtracts 1; U adds 1 to y, D subtracts 1. At the end check both are zero.

The Python you would have written.

```
def judgeCircle(moves):
    x = y = 0
    for m in moves:
        if m == 'U': y += 1
        elif m == 'D': y -= 1
        elif m == 'R': x += 1
        elif m == 'L': x -= 1
    return x == 0 and y == 0
```

In C++. This is the full file. main() at the bottom runs the tests.

```
// 01_robot_return_to_origin.cpp – LeetCode 657. Robot Return to Origin (easy)
#include <cassert>
#include <iostream>
#include <string>

// Net displacement is all that matters: sum the unit vectors and check for (0,0).
bool judgeCircle(std::string moves) {
    int x = 0, y = 0;
    for (const char m : moves) {
        switch (m) {
            case 'U': ++y; break;
            case 'D': --y; break;
            case 'R': ++x; break;
            case 'L': --x; break;
            default: break; // input guarantees only U/D/L/R
        }
    }
    return x == 0 && y == 0;
}

int main() {
    assert(judgeCircle("UD") == true);
    assert(judgeCircle("LL") == false);
    assert(judgeCircle("RRDD") == false);
    assert(judgeCircle("LDRRLRUULR") == false);
    assert(judgeCircle("") == true); // no moves: already at origin
    assert(judgeCircle("UDLR") == true);
    std::cout << "OK 01_robot_return_to_origin.cpp\n";
    return 0;
}
```

Line by line, what is new compared with Python:

- `bool judgeCircle(std::string moves)` says the function returns a `bool` and takes a string.
- `for (const char m : moves)` is Python's `for m in moves`. `const char` is the type of one character; `const` because we only read it.
- `switch (m) { case 'U': ++y; break; ... }` is the C++ way to write a chain of `if m == ...` on a single character. Every case needs `break` or execution falls through into the next case. `default:` is the "else". You could write `if / else if` instead; both are fine.
- `++y` is `y += 1`.

Walk through "RRDD". Start `x=0, y=0`. R: `x=1`. R: `x=2`. D: `y=-1`. D: `y=-2`. End: `x == 0` is false, so return false.

Complexity. One pass over the string: time proportional to its length, constant extra memory.

Robotics. This is dead reckoning at its simplest: integrate the commanded motions to estimate where you are. Real robots do this with wheel encoders and it drifts, which is why they also use sensors and a Kalman filter (chapter 06).

Follow-ups you may get. - *What if moves could be diagonal?* Add two more cases, or store direction vectors in a map from `char` to `(dx, dy)`. - *What if the robot turns instead of moving in absolute directions?* Keep a heading (0 to 3) and rotate it on L/R; that is Robot Bounded In Circle (LeetCode 1041).

2. Flood Fill (LeetCode 733, easy)

The problem in plain words. You are given an image as a grid of integers (each integer is a colour), a starting cell `(sr, sc)`, and a new colour. Recolour the starting cell *and every cell connected to it that has the same colour as the starting cell*. "Connected" means you can reach it by moving up, down, left or right through cells of that same colour. Return the modified image. This is the paint-bucket tool in an image editor.

```
image = 1 1 1      start (1, 1), new colour 2
        1 1 0
        1 0 1
```

The starting cell has colour 1. All the 1s connected to it (not the lone 1 in the bottom-right corner, which is cut off by 0s) become 2:

```
result = 2 2 2
         2 2 0
         2 0 1
```

By hand. Put your finger on the start. Colour it. Look at its four neighbours: any that has the *original* colour, move there and repeat. Stop when nothing next to you has the original colour. That is DFS from A3, where "already visited" is simply "no longer the original colour".

The idea. Write a recursive function `fill(r, c)`: if `(r, c)` is outside the image or is not the original colour, stop. Otherwise set it to the new colour and call `fill` on the four neighbours. Because a filled cell no longer has the original colour, it is never filled twice.

One trap: if the new colour equals the original colour, "not the original colour" is never true and the recursion never stops. Check for that up front and do nothing.

The Python you would have written.

```
def floodFill(image, sr, sc, color):
    rows, cols = len(image), len(image[0])
    original = image[sr][sc]
    if original == color:
        return image          # nothing to do; avoids infinite recursion
    def fill(r, c):
        if not (0 <= r < rows and 0 <= c < cols) or image[r][c] != original:
            return
        image[r][c] = color
        fill(r + 1, c); fill(r - 1, c); fill(r, c + 1); fill(r, c - 1)
    fill(sr, sc)
    return image
```

In C++.

```
// 02_flood_fill.cpp - LeetCode 733. Flood Fill (easy)
#include <cassert>
#include <iostream>
#include <vector>

// Recursive DFS. The recolouring itself is the visited mark: once a cell is
// newColor it no longer equals the original colour and is skipped.
void fill(std::vector<std::vector<int>>& img, int r, int c, int from, int to) {
    const int rows = static_cast<int>(img.size());
    const int cols = static_cast<int>(img[0].size());
    if (r < 0 || r >= rows || c < 0 || c >= cols || img[r][c] != from) return;
    img[r][c] = to;
    fill(img, r + 1, c, from, to);
    fill(img, r - 1, c, from, to);
    fill(img, r, c + 1, from, to);
    fill(img, r, c - 1, from, to);
}

std::vector<std::vector<int>> floodFill(std::vector<std::vector<int>>& image, int sr, int sc, int color) {
    // If the start already has the target colour, filling would never terminate
    // without this guard (every neighbour "still" matches the original colour).
    if (image[sr][sc] != color) fill(image, sr, sc, image[sr][sc], color);
    return image;
}

int main() {
    {
        std::vector<std::vector<int>> img = {{1, 1, 1}, {1, 1, 0}, {1, 0, 1}};
    }
}
```

```

    const std::vector<std::vector<int>> want = {{2, 2, 2}, {2, 2, 0}, {2, 0, 1}};
    assert(floodFill(img, 1, 1, 2) == want);
}
{
    std::vector<std::vector<int>> img = {{0, 0, 0}, {0, 0, 0}};
    const std::vector<std::vector<int>> want = img;
    assert(floodFill(img, 0, 0, 0) == want); // same colour: unchanged, no infinite loop
}
{
    std::vector<std::vector<int>> img = {{5}};
    assert(floodFill(img, 0, 0, 7) == std::vector<std::vector<int>>{{7}});
}
{
    // Diagonal cells are not 4-connected: the two 1s stay separate.
    std::vector<std::vector<int>> img = {{1, 0}, {0, 1}};
    const std::vector<std::vector<int>> want = {{9, 0}, {0, 1}};
    assert(floodFill(img, 0, 0, 9) == want);
}
std::cout << "OK 02_flood_fill.cpp\n";
return 0;
}

```

What is new:

- C++ has no nested functions, so `fill` is a separate function above `floodFill`, and everything it needs (`img`, the original colour `from`, the new colour `to`) is passed as parameters.
- `std::vector<std::vector<int>>& img`: the `&` means "work on the caller's image, do not copy it". Without the `&` every recursive call would paint its own private copy and the caller would see no change. This is the mistake to watch for in every recursive grid function.
- The four neighbour checks and the colour check are combined into one `if (...) return;` at the top. Order matters: the bounds checks come first so `img[r][c]` is never read out of range.
- `floodFill` returns `image` by value at the end; that copies the grid once, which is what LeetCode's signature asks for.

Walk through the example. `from = 1, to = 2`. `fill(1,1)`: it is 1, set to 2. Recurse down `fill(2,1)`: it is 0, stop. Up `fill(0,1)`: `1 → 2`; from there `fill(-1,1)` is out, `fill(1,1)` is now 2 so stop, `fill(0,2)`: `1 → 2`, ..., `fill(0,0)`: `1 → 2`, `fill(1,0)`: `1 → 2`, `fill(2,0)`: `1 → 2`. Everything connected is now 2. The cell `(2,2)` was never reached because every route to it goes through a 0.

Complexity. Each cell is painted at most once, so time is proportional to the number of cells. Memory is the recursion depth, which in the worst case (a snake-shaped region) is also the number of cells.

Robotics. Flood fill from the robot's own cell over the free cells of an occupancy grid gives the region the robot can actually reach. Cells outside it are unreachable goals; cells on its edge next to unknown space are *frontiers*, which is how exploration picks where to go next.

Follow-ups you may get. - *Iterative version?* Replace recursion with a `std::stack<std::pair<int,int>>`: push the start, loop while not empty, pop, paint, push same-colour neighbours. - *8-connected (diagonals count)?* Use the 8-entry `dr/dc` arrays instead of four calls.

3. Number of Islands (LeetCode 200, medium)

The problem in plain words. A grid of characters where '1' is land and '0' is water. An island is a group of '1' cells connected up/down/left/right (diagonal does *not* count). The grid is surrounded by water. Count the islands.

```

1 1 0 0 0
1 1 0 0 0      three islands: the 2x2 block top-left,
0 0 1 0 0      the single cell in the middle,
0 0 0 1 1      and the pair bottom-right.

```

The grid can be up to 300 by 300.

By hand. Scan the grid left to right, top to bottom. The first time you see a '1' that you have not already coloured in, that is a new island: add one to the count, then colour in the *whole* island so you do not count it again. Colouring in the whole island is flood fill from problem 2.

The idea. For every cell: if it is '1', increment the count and run a DFS that turns every connected '1' into '0' ("sink" the island). Later scans then skip those cells. Sinking the island in place is the visited mark; no separate table needed.

The Python you would have written.

```
def numIslands(grid):
    rows, cols = len(grid), len(grid[0])
    def sink(r, c):
        if not (0 <= r < rows and 0 <= c < cols) or grid[r][c] != '1':
            return
        grid[r][c] = '0'
        sink(r + 1, c); sink(r - 1, c); sink(r, c + 1); sink(r, c - 1)
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                sink(r, c)
    return count
```

In C++. This version uses an explicit stack instead of recursion. Read A3 again for why: a 300 by 300 grid of all land would recurse 90,000 levels deep, which is risky. The stack version is the same DFS with the "back up" done by a `std::stack` instead of the call stack.

```
// 03_number_of_islands.cpp - LeetCode 200. Number of Islands (medium)
#include <cassert>
#include <iostream>
#include <stack>
#include <utility>
#include <vector>

// Iterative DFS that sinks each island by overwriting '1' with '0' in place.
// Mutating the input is the cheapest visited set. Copy the grid first if the
// caller still needs the original.
void sink(std::vector<std::vector<char>>& g, int sr, int sc) {
    const int rows = static_cast<int>(g.size());
    const int cols = static_cast<int>(g[0].size());
    constexpr int dr[4] = {-1, 0, 1, 0};
    constexpr int dc[4] = {0, 1, 0, -1};
    std::stack<std::pair<int, int>> st;
    st.push({sr, sc});
    g[sr][sc] = '0';
    while (!st.empty()) {
        auto [r, c] = st.top();
        st.pop();
        for (int k = 0; k < 4; ++k) {
            const int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || g[nr][nc] != '1') continue;
            g[nr][nc] = '0'; // mark when pushing so a cell is never pushed twice
            st.push({nr, nc});
        }
    }
}

int numIslands(std::vector<std::vector<char>>& grid) {
    if (grid.empty() || grid[0].empty()) return 0;
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    int islands = 0;
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c)
```

```

        if (grid[r][c] == '1') {
            ++islands;
            sink(grid, r, c);
        }
    }
    return islands;
}

int main() {
    {
        std::vector<std::vector<char>> g = {
            {'1', '1', '1', '1', '0'},
            {'1', '1', '0', '1', '0'},
            {'1', '1', '0', '0', '0'},
            {'0', '0', '0', '0', '0'}};
        assert(numIslands(g) == 1);
    }
    {
        std::vector<std::vector<char>> g = {
            {'1', '1', '0', '0', '0'},
            {'1', '1', '0', '0', '0'},
            {'0', '0', '1', '0', '0'},
            {'0', '0', '0', '1', '1'}};
        assert(numIslands(g) == 3); // diagonals do not connect
    }
    {
        std::vector<std::vector<char>> g;
        assert(numIslands(g) == 0);
    }
    {
        std::vector<std::vector<char>> g = {{'0'}};
        assert(numIslands(g) == 0);
    }
    {
        // Large all-land grid: recursion would be 250k deep; the stack version is fine.
        std::vector<std::vector<char>> g(500, std::vector<char>(500, '1'));
        assert(numIslands(g) == 1);
    }
    std::cout << "OK 03_number_of_islands.cpp\n";
    return 0;
}

```

What is new:

- `std::stack<std::pair<int,int>> st;` is a stack of (row, col). `st.push`, `st.top`, `st.pop` match queue's `push`, `front`, `pop`. The only difference from the BFS loop in A2 is `top()` instead of `front()`, which makes it last-in-first-out and therefore a DFS.
- `g[nr][nc] = '0'` happens *when pushing*. Same rule as BFS: mark on push so each cell is pushed once.
- `if (grid.empty() || grid[0].empty()) return 0;` guards against an empty grid before `grid[0]` is touched. `grid[0]` on an empty vector is undefined behaviour in C++, not an exception.
- The nested for loops without braces around a single `if` are normal C++ style; add braces if you find it clearer.

Walk through the example. Scan: (0,0) is '1' → count = 1, sink the 2x2 block (four cells become '0'). Continue scanning: (0,1), (1,0), (1,1) are now '0', skip. (2,2) is '1' → count = 2, sink it (just one cell). (3,3) is '1' → count = 3, sink (3,3) and (3,4). End of grid: 3.

Complexity. Every cell is scanned once and sunk at most once: time proportional to rows × cols. The stack holds at most one entry per cell.

Robotics. This is connected-component labelling on a map. Grouping obstacle cells into distinct obstacles, grouping frontier cells into frontier regions for exploration, and clustering laser hits into objects all use exactly this loop.

Follow-ups you may get. - *Do not modify the input.* Keep a `std::vector<std::vector<bool>>` visited (or a flat `std::vector<char>` of size rows × cols) and check that instead of overwriting. - *Largest island instead of count?* Have sink return how many cells it sank; keep the max. - *Islands appear one at a time and you must report the count after each?* That is Number of Islands II (LeetCode 305), solved with union-find (chapter 04).

4. Rotting Oranges (LeetCode 994, medium)

The problem in plain words. A grid where each cell is 0 (empty), 1 (a fresh orange) or 2 (a rotten orange). Every minute, every fresh orange that is directly next to a rotten one (up/down/left/right) becomes rotten. Return the number of minutes until no fresh orange is left. If some fresh orange can never be reached by the rot, return -1.

minute 0	minute 1	minute 2	minute 3	minute 4	
2 1 1	2 2 1	2 2 2	2 2 2	2 2 2	
1 1 0	2 1 0	2 2 0	2 2 0	2 2 0	
0 1 1	0 1 1	0 1 1	0 2 1	0 2 2	-> answer 4

The orange at (2,2) is the last to rot, at minute 4. If the grid were 2 1 1 / 0 1 1 / 1 0 1, the orange at (2,0) is walled off by zeros and never rots: answer -1. If there are no fresh oranges at all, the answer is 0.

By hand. Look at the minute-by-minute pictures. The rot spreads outward from *all* rotten oranges at once, one ring per minute. That is exactly the BFS ripple from A2, with two twists from the variants: it starts from *many* sources at once (every rotten orange), and we want to *count rings* (minutes) rather than record a distance per cell.

The idea. 1. Scan the grid. Push every rotten orange into the queue. Count the fresh ones. 2. Loop one ring at a time: snapshot the queue size, pop that many cells, and for each fresh neighbour mark it rotten (write 2 into the grid), decrement the fresh count, push it. After the ring, add one minute. 3. Stop when the queue is empty or no fresh oranges remain. If fresh is zero, return the minutes; otherwise -1.

The check `fresh > 0` in the loop condition stops us counting an extra minute in which nothing rots.

The Python you would have written.

```
from collections import deque

def orangesRotting(grid):
    rows, cols = len(grid), len(grid[0])
    q = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2: q.append((r, c))
            elif grid[r][c] == 1: fresh += 1
    minutes = 0
    while q and fresh > 0:
        for _ in range(len(q)):                # one ring
            r, c = q.popleft()
            for dr, dc in ((-1,0),(0,1),(1,0),(0,-1)):
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    q.append((nr, nc))
            minutes += 1
    return minutes if fresh == 0 else -1
```

In C++.

```
// 04_rotting_oranges.cpp – LeetCode 994. Rotting Oranges (medium) – multi-source BFS
#include <cassert>
#include <iostream>
```

```

#include <queue>
#include <utility>
#include <vector>

// Every rotten orange is a BFS source at time 0. Process the queue level by
// level; each level is one minute. The answer is the number of levels that
// actually rotted something, and -1 if any fresh orange survives.
int orangesRotting(std::vector<std::vector<int>>& grid) {
    const int rows = static_cast<int>(grid.size());
    const int cols = rows ? static_cast<int>(grid[0].size()) : 0;
    std::queue<std::pair<int, int>> q;
    int fresh = 0;
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c) {
            if (grid[r][c] == 2) q.push({r, c});
            else if (grid[r][c] == 1) ++fresh;
        }
    constexpr int dr[4] = {-1, 0, 1, 0};
    constexpr int dc[4] = {0, 1, 0, -1};
    int minutes = 0;
    while (!q.empty() && fresh > 0) {
        const int levelSize = static_cast<int>(q.size()); // snapshot: the queue grows inside
        for (int i = 0; i < levelSize; ++i) {
            auto [r, c] = q.front();
            q.pop();
            for (int k = 0; k < 4; ++k) {
                const int nr = r + dr[k], nc = c + dc[k];
                if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || grid[nr][nc] != 1) continue;
                grid[nr][nc] = 2; // the grid itself is the visited set
                --fresh;
                q.push({nr, nc});
            }
        }
        ++minutes;
    }
    return fresh == 0 ? minutes : -1;
}

int main() {
    {
        std::vector<std::vector<int>> g = {{2, 1, 1}, {1, 1, 0}, {0, 1, 1}};
        assert(orangesRotting(g) == 4);
    }
    {
        std::vector<std::vector<int>> g = {{2, 1, 1}, {0, 1, 1}, {1, 0, 1}};
        assert(orangesRotting(g) == -1); // bottom-left orange is isolated
    }
    {
        std::vector<std::vector<int>> g = {{0, 2}};
        assert(orangesRotting(g) == 0); // nothing fresh: zero minutes
    }
    {
        std::vector<std::vector<int>> g = {{1}};
        assert(orangesRotting(g) == -1); // fresh but no source
    }
    {
        std::vector<std::vector<int>> g = {{2, 1, 1, 1, 2}};
        assert(orangesRotting(g) == 2); // two sources meet in the middle
    }
    std::cout << "OK 04_rotting_oranges.cpp\n";
    return 0;
}

```

What is new:

- `const int cols = rows ? static_cast<int>(grid[0].size()) : 0;` is a ternary: "if rows is non-zero, use `grid[0].size()`, else 0". It avoids touching `grid[0]` on an empty grid.

- `const int levelSize = static_cast<int>(q.size());` is Python's `for _ in range(len(q))`. It must be taken *before* the inner loop because the queue grows during it.
- `grid[nr][nc] = 2;` is both "rot it" and "mark it visited". No separate visited table.
- `return fresh == 0 ? minutes : -1;` is Python's `minutes if fresh == 0 else -1`.

Walk through the example. Queue starts with `[(0,0)]`, `fresh = 6`. Minute 1 ring: pop `(0,0)`, rot `(0,1)` and `(1,0)`, `fresh = 4`, `minutes = 1`. Minute 2 ring: pop `(0,1)`, rot `(0,2)`; pop `(1,0)`, rot `(1,1)`; `fresh = 2`, `minutes = 2`. Minute 3 ring: pop `(0,2)`, nothing new; pop `(1,1)`, rot `(2,1)`; `fresh = 1`, `minutes = 3`. Minute 4 ring: pop `(2,1)`, rot `(2,2)`; `fresh = 0`, `minutes = 4`. Loop ends because `fresh` is 0. Return 4.

Complexity. Every cell enters the queue at most once: time and memory proportional to `rows × cols`.

Robotics. This is the wavefront planner. Seed the queue with the goal cell (or several goals), BFS outward over free cells, and every cell ends up labelled with its distance to the nearest goal. The robot then simply steps to whichever neighbour has the smaller number. The "minutes" here are the wavefront's distance rings.

Follow-ups you may get. - *Why not run BFS from each rotten orange separately and take the minimum?* That is `(number of rotten) × (grid size)`; pushing all sources at once gives the same answer in one pass. - *What if you also need to know which orange rotted each cell?* Store a source id alongside the cell when you push it, or keep a parallel `source` grid. - *Why mark when pushing rather than when popping?* So a fresh orange next to two rotten ones is pushed once, not twice, and the ring sizes stay correct.

5. Shortest Path in Binary Matrix (LeetCode 1091, medium)

The problem in plain words. You are given a square grid of 0s and 1s, `n` rows by `n` columns. 0 is a free cell, 1 is blocked. You start at the top-left cell `(0, 0)` and want to reach the bottom-right cell `(n-1, n-1)`. From a cell you may move to any of its **eight** neighbours (up, down, left, right, and the four diagonals), as long as the neighbour is free. Return the length of the shortest such path, where length is the **number of cells on the path, including the start and the goal**. If there is no path, or the start or goal is blocked, return `-1`.

```
grid =  0 0 0      One shortest path: (0,0) -> (0,1) -> (1,2) -> (2,2).
        1 1 0      That is 3 moves but 4 cells, so the answer is 4.
        1 1 0
```

Note the second move, `(0,1) -> (1,2)`: that is a diagonal step, down and right at once. Without diagonals the answer would be 5 (`(0,0) (0,1) (0,2) (1,2) (2,2)`).

More examples and the edge cases:

```
0 1      one diagonal step from (0,0) to (1,1): 2 cells    -> 2
1 0

1 0 0
1 1 0      the start cell is blocked                        -> -1
1 1 0

0          a 1x1 grid: start is the goal, the path is 1 cell -> 1

0 0 0
0 1 0      go round the blocked centre: (0,0)(0,1)(1,2)(2,2) -> 4
0 0 0
```

`n` is at most 100, so the grid has at most 10,000 cells.

Cells, not steps. Most path problems count moves. This one counts cells, which is always moves + 1. So the start cell alone has "length 1", and a 1x1 grid returns 1, not 0. The whole difference in the code is one character: the start gets distance 1 instead of 0.

By hand. This is the ripple from A2 with two changes. First, the ripple spreads to eight neighbours instead of four, so it also grows diagonally. Second, we do not need the distance of every cell; we can stop the moment the ripple touches the goal. Drop the stone at (0,0) with ring number 1. Ring 2 is every free neighbour of the start. Ring 3 is every free unreached neighbour of ring 2, and so on. The ring number of the goal, when it first appears, is the answer.

The eight neighbours. For the cell at (r, c):

```
(r-1, c-1) (r-1, c) (r-1, c+1)
(r, c-1) (r, c) (r, c+1)
(r+1, c-1) (r+1, c) (r+1, c+1)
```

Every neighbour is "row plus one of {-1, 0, +1}, column plus one of {-1, 0, +1}", except the middle one where both are 0 (that is the cell itself). So instead of an 8-entry table you can write two nested loops over -1..1 and skip the (0, 0) case. Both ways are fine; the solution below uses the loops.

The idea. 1. If the start or the goal is blocked, return -1 immediately. 2. Make a `dist` table of zeros. Zero means "not reached yet" (a real distance is never zero here, because the start is already 1). 3. Set `dist[0][0] = 1` and push (0,0). 4. Loop: pop the front. If it is the goal, return its distance. Otherwise, for each of the eight neighbours that is inside the grid, free, and unreached, write `dist + 1` and push it. 5. If the queue runs dry, the goal was never reached: return -1.

Why returning at the first pop is safe. BFS pops cells in order of distance: all the 2s come out before any 3, all the 3s before any 4. So the first time the goal is popped, no cell with a smaller distance is still waiting, and its distance is the smallest possible. You could also return at the moment you *push* the goal, since in BFS the distance written on push is already final; that saves finishing one ring. Returning on pop is the habit worth keeping, because it is the version that still works for Dijkstra and A* in Part C, where a pushed distance can later be improved and only the pop is final.

The Python you would have written.

```
from collections import deque

def shortestPathBinaryMatrix(grid):
    n = len(grid)
    if grid[0][0] != 0 or grid[n - 1][n - 1] != 0:
        return -1
    dist = [[0] * n for _ in range(n)]          # 0 = not reached
    dist[0][0] = 1                             # path length counts cells
    q = deque([(0, 0)])
    while q:
        r, c = q.popleft()
        if r == n - 1 and c == n - 1:
            return dist[r][c]
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                if dr == 0 and dc == 0:
                    continue
                nr, nc = r + dr, c + dc
                if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0 and dist[nr][nc] == 0:
                    dist[nr][nc] = dist[r][c] + 1
                    q.append((nr, nc))
    return -1
```

In C++.

```
// 05_shortest_path_binary_matrix.cpp - LeetCode 1091. Shortest Path in Binary Matrix (medium)
// 8-connected BFS; path length counts cells, so the start contributes 1.
#include <cassert>
```

```

#include <iostream>
#include <queue>
#include <utility>
#include <vector>

int shortestPathBinaryMatrix(std::vector<std::vector<int>>& grid) {
    const int n = static_cast<int>(grid.size());
    if (n == 0 || grid[0][0] != 0 || grid[n - 1][n - 1] != 0) return -1;
    std::vector<std::vector<int>> dist(n, std::vector<int>(n, 0)); // 0 = unvisited
    std::queue<std::pair<int, int>> q;
    dist[0][0] = 1;
    q.push({0, 0});
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        if (r == n - 1 && c == n - 1) return dist[r][c]; // first pop of the goal is optimal in BFS
        // All eight neighbours, generated by a double loop instead of a table.
        for (int dr = -1; dr <= 1; ++dr)
            for (int dc = -1; dc <= 1; ++dc) {
                if (dr == 0 && dc == 0) continue;
                const int nr = r + dr, nc = c + dc;
                if (nr < 0 || nr >= n || nc < 0 || nc >= n || grid[nr][nc] != 0 || dist[nr][nc] != 0) continue;
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
    }
    return -1;
}

int main() {
    {
        std::vector<std::vector<int>> g = {{0, 1}, {1, 0}};
        assert(shortestPathBinaryMatrix(g) == 2); // one diagonal step
    }
    {
        std::vector<std::vector<int>> g = {{0, 0, 0}, {1, 1, 0}, {1, 1, 0}};
        assert(shortestPathBinaryMatrix(g) == 4);
    }
    {
        std::vector<std::vector<int>> g = {{1, 0, 0}, {1, 1, 0}, {1, 1, 0}};
        assert(shortestPathBinaryMatrix(g) == -1); // start blocked
    }
    {
        std::vector<std::vector<int>> g = {{0}};
        assert(shortestPathBinaryMatrix(g) == 1); // start == goal
    }
    {
        std::vector<std::vector<int>> g = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
        assert(shortestPathBinaryMatrix(g) == 4); // around the centre: (0,0)(0,1)(1,2)(2,2) or mirror
    }
    std::cout << "OK 05_shortest_path_binary_matrix.cpp\n";
    return 0;
}

```

What is new:

- `if (n == 0 || grid[0][0] != 0 || grid[n - 1][n - 1] != 0) return -1;` checks the empty grid *first*. `||` stops at the first true condition, so when `n == 0` the `grid[0][0]` part is never evaluated. That order matters: reading `grid[0][0]` on an empty vector is undefined behaviour in C++, not an `IndexError`.
- `std::vector<std::vector<int>> dist(n, std::vector<int>(n, 0));` is `[[0] * n for _ in range(n)]`.
- `for (int dr = -1; dr <= 1; ++dr) for (int dc = -1; dc <= 1; ++dc) { ... }` is the pair of `for` `dr` in `(-1, 0, 1)` loops. The outer `for` has no braces because its whole body is the single inner `for`. If you would rather have a table, this is the same thing:

```
cpp const int dr[8] = {-1, -1, -1, 0, 0, 1, 1, 1}; const int dc[8] = {-1, 0, 1, -1, 1, -1, 0, 1}; for
(int k = 0; k < 8; ++k) { /* nr = r + dr[k], nc = c + dc[k] */ }
```

- `const int nr = r + dr, nc = c + dc;` declares two `const` ints in one statement, exactly like `nr, nc = r + dr, c + dc`.

- The one long `if (... ) continue;` bundles four checks: outside the grid, blocked, already reached. The bounds checks come first for the same reason as in A1: `grid[nr][nc]` must not be read until you know `nr` and `nc` are valid.
- `return dist[r][c];` inside the `while` leaves the function straight away, as in Python.

Walk through the example (0 0 0 / 1 1 0 / 1 1 0). `dist[0][0] = 1`, queue `[(0,0)]`.

```
pop (0,0)  not the goal. Of its 8 neighbours only (0,1) is inside and free:
           dist[0][1] = 2.                               queue [(0,1)]
pop (0,1)  free unreached neighbours: (0,2) and (1,2) diagonally below-right:
           both get 3.                                   queue [(0,2), (1,2)]
pop (0,2)  (1,1) blocked, (1,2) already 3, nothing new. queue [(1,2)]
pop (1,2)  (2,2) below gets 4; (2,1) blocked.           queue [(2,2)]
pop (2,2)  this is the goal: return dist[2][2] = 4.
```

Final dist table (0 means never reached):

```
1 2 3
0 0 3
0 0 4
```

Complexity. Every cell is pushed at most once and looks at 8 neighbours, so time is proportional to the number of cells: $O(n^2)$. The `dist` table and the queue are also $O(n^2)$.

Robotics. An 8-connected search over a binary occupancy grid is the simplest global planner. Notice one thing this problem allows that a real planner must not: the 0 1 / 1 0 example squeezes diagonally between two blocked cells that touch only at a corner. A robot has width and would clip both. The A* project in Part C refuses that move on purpose.

Follow-ups you may get. - *Diagonal moves should cost $\sqrt{2}$, not 1.* Then steps no longer have equal cost and plain BFS is wrong; use Dijkstra (a priority queue keyed on distance) or A, Part C. - *Why BFS and not DFS? DFS finds a path, not the shortest one. Only BFS explores in order of distance.* - Return the path itself, not just its length.* Store a `parent` cell for each cell when you push it, then walk back from the goal to the start and reverse.

6. 01 Matrix (LeetCode 542, medium)

The problem in plain words. You are given a grid of 0s and 1s. For every cell, find the distance to the nearest 0, counting a move up, down, left or right as 1 (no diagonals). Return a grid of the same shape holding those distances. Cells that are 0 have distance 0 to themselves. There is always at least one 0 in the grid.

```
input:  0 0 0      output: 0 0 0
        0 1 0        0 1 0
        1 1 1        1 2 1
```

The middle 1 at (1,1) touches a 0 directly, so its distance is 1. The bottom-middle cell (2,1) is two moves from the nearest 0 (for example (2,1) -> (1,1) -> (0,1) or (2,1) -> (2,0) -> (1,0)), so it gets 2.

A second example, with only one zero:

```
input:  0 1 1      output: 0 1 2
        1 1 1        1 2 3
```

Here every cell is just its Manhattan distance to the corner, because there is only one source. A 1x1 grid `[[0]]` returns `[[0]]`. The grid has at most 10,000 cells.

By hand. If there were a single 0, you would run the ripple from A2 starting there and the `dist` table is the answer. With several zeros, imagine dropping a stone into the pond at every zero at the same instant. All the ripples spread at the same speed. A cell is first touched by whichever ripple started closest to it, and the ring number at that moment is the distance to the nearest zero. That is Variant 2 in A2, multi-source BFS: push every zero into the queue with distance 0 before the loop starts, and run the ordinary BFS.

Why "all at once" gives the nearest zero. In single-source BFS, the first time a cell is reached is by a shortest path from the one source. With all zeros in the queue at distance 0, the queue still pops cells in order of distance: all the 0s, then every cell at distance 1 from *some* zero, then every cell at distance 2 from *some* zero. A cell at true distance d from its nearest zero is reached in ring d and cannot be reached earlier, because that would mean a zero closer than d . If it helps, picture one imaginary super-source joined to every zero by a free edge: single-source BFS from it gives exactly the same rings.

The naive way, and why it is slow. The first instinct is "for each 1, run a BFS until I hit a 0". Each of those BFS runs can scan the whole grid, and there can be nearly $\text{rows} \times \text{cols}$ ones, so the total is $(\text{rows} \times \text{cols})^2$. On a 10,000-cell grid that is up to 10^8 cell visits. The multi-source version visits each cell once: 10^4 . Same answer, ten thousand times less work.

The idea. 1. Make `dist` all -1 ("not reached"). 2. Scan the grid. Every 0 gets `dist = 0` and goes in the queue. 3. Ordinary BFS: pop, and for each in-bounds neighbour with `dist == -1`, write `dist + 1` and push. 4. Return `dist`.

Notice there is no "is this neighbour a 1?" check. Every zero is already marked with distance 0 before the loop, so the `dist == -1` test alone skips zeros and reached cells, and only unreached ones remain.

The Python you would have written.

```
from collections import deque

def updateMatrix(mat):
    rows, cols = len(mat), len(mat[0])
    dist = [[-1] * cols for _ in range(rows)]
    q = deque()
    for r in range(rows):
        for c in range(cols):
            if mat[r][c] == 0:
                dist[r][c] = 0
                q.append((r, c))          # every zero is a source
    while q:
        r, c = q.popleft()
        for dr, dc in ((-1, 0), (0, 1), (1, 0), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))
    return dist
```

In C++.

```
// 06_01_matrix.cpp - LeetCode 542. 01 Matrix (medium) - distance transform via multi-source BFS
#include <cassert>
#include <iostream>
#include <queue>
#include <utility>
#include <vector>

// Seed the queue with every 0 at distance 0, then expand. The first time BFS
// reaches a cell is from its nearest zero, so the distance is final on push.
std::vector<std::vector<int>> updateMatrix(std::vector<std::vector<int>>& mat) {
    const int rows = static_cast<int>(mat.size());
    const int cols = rows ? static_cast<int>(mat[0].size()) : 0;
    std::vector<std::vector<int>> dist(rows, std::vector<int>(cols, -1));
    std::queue<std::pair<int, int>> q;
    for (int r = 0; r < rows; ++r)
```

```

    for (int c = 0; c < cols; ++c)
        if (mat[r][c] == 0) {
            dist[r][c] = 0;
            q.push({r, c});
        }
constexpr int dr[4] = {-1, 0, 1, 0};
constexpr int dc[4] = {0, 1, 0, -1};
while (!q.empty()) {
    auto [r, c] = q.front();
    q.pop();
    for (int k = 0; k < 4; ++k) {
        const int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || dist[nr][nc] != -1) continue;
        dist[nr][nc] = dist[r][c] + 1;
        q.push({nr, nc});
    }
}
return dist;
}

int main() {
{
    std::vector<std::vector<int>>> m = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
    const std::vector<std::vector<int>>> want = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
    assert(updateMatrix(m) == want);
}
{
    std::vector<std::vector<int>>> m = {{0, 0, 0}, {0, 1, 0}, {1, 1, 1}};
    const std::vector<std::vector<int>>> want = {{0, 0, 0}, {0, 1, 0}, {1, 2, 1}};
    assert(updateMatrix(m) == want);
}
{
    std::vector<std::vector<int>>> m = {{0}};
    assert(updateMatrix(m) == std::vector<std::vector<int>>>{{0}});
}
{
    // A single zero in the corner: distances are Manhattan distances to it.
    std::vector<std::vector<int>>> m = {{0, 1, 1}, {1, 1, 1}};
    const std::vector<std::vector<int>>> want = {{0, 1, 2}, {1, 2, 3}};
    assert(updateMatrix(m) == want);
}
std::cout << "OK 06_01_matrix.cpp\n";
return 0;
}

```

What is new:

- `std::vector<std::vector<int>>> updateMatrix(...)` returns a whole grid. `return dist;` at the end hands the table back to the caller; C++ moves it rather than copying, so this is cheap.
- `const int cols = rows ? static_cast<int>(mat[0].size()) : 0;` is the same empty-grid guard you saw in Rotting Oranges: only touch `mat[0]` if there is a row 0.
- `constexpr int dr[4] = {-1, 0, 1, 0};` is the direction table from A1. `constexpr` is `const` with the extra promise that the value is known at compile time. For a table of literals it behaves exactly like `const`; either spelling is fine here.
- The two scanning `for` loops and the `if` have no braces around the single statement they contain, and then braces around the two-line body. That is normal C++ style; add braces everywhere if you prefer.
- `auto [r, c] = q.front(); q.pop();` is `r, c = q.popleft();`, as in A2.
- The `main()` tests compare two `std::vector<std::vector<int>>>` with `==`. That works out of the box: vectors compare element by element, like Python lists.

Walk through the first example (0 0 0 / 0 1 0 / 1 1 1). The scan pushes the five zeros in reading order: queue [(0,0), (0,1), (0,2), (1,0), (1,2)], all with dist 0.

```

pop (0,0)  neighbours (0,1) and (1,0) already 0.      nothing new
pop (0,1)  down (1,1) unreachable -> 1.               queue [.., (1,1)]
pop (0,2)  down (1,2) already 0.                     nothing new
pop (1,0)  down (2,0) unreachable -> 1. right (1,1) done. queue [.., (1,1), (2,0)]
pop (1,2)  down (2,2) unreachable -> 1.               queue [(1,1), (2,0), (2,2)]
pop (1,1)  down (2,1) unreachable -> 2.               queue [(2,0), (2,2), (2,1)]
pop (2,0)  right (2,1) already 2.                     nothing new
pop (2,2)  left (2,1) already 2.                     nothing new
pop (2,1)  everything around is reached.               queue empty, done

```

The table now reads 0 0 0 / 0 1 0 / 1 2 1. Note (2,1) was pushed once, by (1,1), even though (2,0) and (2,2) also touch it: they arrived later and found it already marked.

Complexity. Every cell enters the queue exactly once and looks at 4 neighbours, so time is proportional to rows × cols: $O(\text{rows} \times \text{cols})$. Memory is the dist table plus the queue, also $O(\text{rows} \times \text{cols})$.

Robotics. This is the *distance transform*, also called brushfire, and it is how a costmap's inflation layer is built: every occupied cell is a source, BFS gives every free cell its distance to the nearest obstacle, and that distance is turned into a cost so the planner keeps the robot away from walls. The cells where two wavefronts meet (equal distance to two different obstacles) form the Voronoi diagram, which some planners follow as the "safest corridor" through a map.

Follow-ups you may get. - *Can you do it without a queue?* Yes, with two DP sweeps: top-left to bottom-right taking $\min(\text{above}, \text{left}) + 1$, then bottom-right to top-left taking $\min(\text{below}, \text{right}) + 1$. Same $O(\text{rows} \times \text{cols})$, no queue, but it only works for Manhattan distance. - *Euclidean distance instead of Manhattan?* Steps no longer have equal cost, so use Dijkstra from all sources, or a proper Euclidean distance transform (Felzenszwalb's algorithm is linear time). - *What if there are no zeros at all?* LeetCode promises at least one. Without it the queue starts empty and every cell stays -1, which is a reasonable "unreachable" answer to return or to document.

7. Unique Paths II (LeetCode 63, medium)

The problem in plain words. A robot sits in the top-left cell of a grid with rows rows and cols columns and wants to reach the bottom-right cell. It can only move **right** or **down**, one cell at a time. Some cells contain an obstacle (1); the rest are free (0). The robot cannot enter an obstacle cell. Count how many different paths lead from the top-left to the bottom-right. Two paths are different if they differ in at least one cell.

```

grid =  0 0 0      . . .      Only two paths avoid the centre:
        0 1 0      . # .      right, right, down, down
        0 0 0      . . .      down, down, right, right
                                     -> 2

```

Without the obstacle there would be 6 paths (the ways table in A5 gives 6 for a 3x3 grid). Edge cases: if the start or the goal is an obstacle the answer is 0; a 1x1 free grid has one path (stand still), answer 1; a full row or full column of obstacles blocks everything, answer 0. The grid is at most 100 by 100, and the final answer is promised to fit in a 32-bit int.

By hand. This is the grid DP from A5, with one extra rule. Write in each cell the number of ways to reach it. The number for a cell is (ways to reach the cell above) + (ways to reach the cell to the left), because the last move into it came from one of those two. An obstacle cell gets 0: you cannot stand there, so no path passes through it. Fill the table row by row, left to right. The example fills in like this:

```

grid:      . . .      ways:    1  1  1
          . # .          1  0  1
          . . .          1  1  2

```

Row 0 is all 1s (only one way along the top edge). In row 1, (1,0) is 1 (from above), (1,1) is the obstacle so 0, and (1,2) = above 1 + left 0 = 1. In row 2, (2,0) = 1, (2,1) = above 0 + left 1 = 1, (2,2) = above 1 + left 1 = 2. The bottom-right cell holds the answer.

The idea. 1. If the start cell is an obstacle, return 0. 2. Define `ways[r][c]` = number of right/down paths from (0,0) to (r,c). 3. `ways[0][0] = 1`. For every other cell: if it is an obstacle, 0; otherwise `ways[r-1][c] + ways[r][c-1]`, treating a missing neighbour (off the top or left edge) as 0. 4. Fill row by row, left to right, so both the cell above and the cell to the left are ready when you need them. Return `ways[rows-1][cols-1]`.

The Python you would have written. This is the two-dimensional table exactly as drawn.

```
def uniquePathsWithObstacles(obstacleGrid):
    rows, cols = len(obstacleGrid), len(obstacleGrid[0])
    if obstacleGrid[0][0] == 1:
        return 0
    ways = [[0] * cols for _ in range(rows)]
    ways[0][0] = 1
    for r in range(rows):
        for c in range(cols):
            if obstacleGrid[r][c] == 1:
                ways[r][c] = 0
                continue
            if r > 0:
                ways[r][c] += ways[r - 1][c]      # from above
            if c > 0:
                ways[r][c] += ways[r][c - 1]      # from the left
    return ways[rows - 1][cols - 1]
```

Squashing the table to one row. Look at what row 2 of the table needs: only row 1 (the cell above) and row 2 itself (the cell to the left). Row 0 is never looked at again. So you do not need to keep the whole table, just one row that you overwrite in place. Call it `dp`. When you start processing row `r`, `dp[c]` still holds row `r-1`'s value for column `c`, which is exactly "the cell above". Sweep `c` from left to right: by the time you reach column `c`, `dp[c-1]` has already been updated to row `r`, which is exactly "the cell to the left". So the update is one line:

```
dp[c] = dp[c] + dp[c-1]
      ^^^^^  ^^^^^^^
      above  left (already updated this row)
```

An obstacle sets `dp[c] = 0`, and column 0 has no left neighbour so it keeps its "from above" value unchanged. The C++ file uses this rolling row. Here is the trace of `dp` for the example, one line per row:

start	<code>dp = [1, 0, 0]</code>	(<code>dp[0] = 1</code> is the start cell)
after row 0	<code>dp = [1, 1, 1]</code>	1, then 0+1, then 0+1
after row 1	<code>dp = [1, 0, 1]</code>	1 from above; obstacle -> 0; 1 + 0
after row 2	<code>dp = [1, 1, 2]</code>	1 from above; 0 + 1; 1 + 1
answer = <code>dp[2]</code>	<code>= 2</code>	

Compare each line with a row of the 2-D table above: they are the same numbers.

In C++.

```
// 07_unique_paths_ii.cpp - LeetCode 63. Unique Paths II (medium) - DP with obstacles
#include <cassert>
#include <iostream>
#include <vector>

// dp[c] = number of ways to reach (current row, c) moving only right/down.
// Rolling a single row: dp[c] already holds "from above", add dp[c-1] "from left".
// An obstacle zeroes its cell so nothing flows through it.
int uniquePathsWithObstacles(std::vector<std::vector<int>>& obstacleGrid) {
    const int rows = static_cast<int>(obstacleGrid.size());
    const int cols = rows ? static_cast<int>(obstacleGrid[0].size()) : 0;
    if (rows == 0 || cols == 0 || obstacleGrid[0][0] == 1) return 0;
```

```

std::vector<long long> dp(cols, 0); // long long: counts can exceed int mid-grid even if the answer fits
dp[0] = 1;
for (int r = 0; r < rows; ++r)
    for (int c = 0; c < cols; ++c) {
        if (obstacleGrid[r][c] == 1) dp[c] = 0;
        else if (c > 0) dp[c] += dp[c - 1];
    }
return static_cast<int>(dp[cols - 1]);
}

int main() {
{
    std::vector<std::vector<int>> g = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
    assert(uniquePathsWithObstacles(g) == 2);
}
{
    std::vector<std::vector<int>> g = {{0, 1}, {0, 0}};
    assert(uniquePathsWithObstacles(g) == 1);
}
{
    std::vector<std::vector<int>> g = {{1}};
    assert(uniquePathsWithObstacles(g) == 0); // start blocked
}
{
    std::vector<std::vector<int>> g = {{0}};
    assert(uniquePathsWithObstacles(g) == 1); // 1x1 free: one empty path
}
{
    std::vector<std::vector<int>> g = {{0, 0}, {1, 1}, {0, 0}};
    assert(uniquePathsWithObstacles(g) == 0); // a full wall row blocks everything
}
{
    // 3x7 without obstacles is the classic Unique Paths answer, C(8,2) = 28.
    std::vector<std::vector<int>> g(3, std::vector<int>(7, 0));
    assert(uniquePathsWithObstacles(g) == 28);
}
std::cout << "OK 07_unique_paths_ii.cpp\n";
return 0;
}

```

What is new:

- `std::vector<long long> dp(cols, 0);` is the single rolling row, `[0] * cols`. The element type is `long long`, a 64-bit integer, instead of `int`. See the next paragraph for why.
- `dp[0] = 1;` is the start cell. There is no separate handling of row 0: the loop treats row 0 like any other row, and because `dp` starts as `[1, 0, 0, ...]`, sweeping left to right turns it into `[1, 1, 1, ...]`, which is the correct top row.
- `if (obstacleGrid[r][c] == 1) dp[c] = 0; else if (c > 0) dp[c] += dp[c - 1];` is the whole transition. `dp[c] += dp[c - 1]` is "above plus left" in place. For `c == 0` nothing happens, so the value carries down from the row above. If column 0 has an obstacle, `dp[0]` becomes 0 and stays 0 for every row below, which is right: nothing can get past it.
- `return static_cast<int>(dp[cols - 1]);` converts back to `int` because that is the return type LeetCode asks for. The cast says "I know this narrows, and I have checked it fits".

Why long long and not int. In Python an integer grows as large as it needs to. In C++ an `int` holds at most 2,147,483,647, and if a sum goes past that the behaviour is undefined: typically it silently wraps to a negative number, and nothing tells you. LeetCode promises the *final* answer fits in an `int`, but it says nothing about the cells in the middle of the table. A large open region whose every exit to the goal is walled off can hold path counts far bigger than the answer. `long long` goes up to about 9.2×10^{18} , four thousand million times more headroom, which covers every grid you will realistically be tested on. Get in the habit: when a DP table holds counts or sums, make it `long long` unless you have a reason not to.

Walk through the example (0 0 0 / 0 1 0 / 0 0 0). rows = cols = 3, start free, `dp = [1, 0, 0]`.

r=0 c=0 free, c==0: nothing.	dp = [1, 0, 0]
r=0 c=1 free: dp[1] += dp[0] -> 1.	dp = [1, 1, 0]
r=0 c=2 free: dp[2] += dp[1] -> 1.	dp = [1, 1, 1]

```

r=1 c=0 free, c==0: nothing (1 from above) dp = [1, 1, 1]
r=1 c=1 obstacle: dp[1] = 0. dp = [1, 0, 1]
r=1 c=2 free: dp[2] += dp[1] -> 1 + 0 = 1. dp = [1, 0, 1]
r=2 c=0 free, c==0: nothing. dp = [1, 0, 1]
r=2 c=1 free: dp[1] += dp[0] -> 0 + 1 = 1. dp = [1, 1, 1]
r=2 c=2 free: dp[2] += dp[1] -> 1 + 1 = 2. dp = [1, 1, 2]
return dp[2] = 2

```

Complexity. One visit per cell with constant work: time proportional to rows $\times$ cols, $O(\text{rows} \times \text{cols})$. Memory is one row: $O(\text{cols})$, instead of $O(\text{rows} \times \text{cols})$ for the full table.

Robotics. Nobody counts robot paths for its own sake, but the *sweep* is everywhere: fill a table in an order where each cell's inputs are already done. Value iteration on a grid world does exactly this when moves are restricted to one direction, and dynamic time warping, used to align a recorded trajectory against a reference one, is the same table with `min` in place of `+`. The rolling-row trick is also how you keep such tables small on an embedded board.

Follow-ups you may get. - *No obstacles: is there a formula?* Yes. Every path is rows-1 downs and cols-1 rights in some order, so the count is the binomial coefficient $C(\text{rows} + \text{cols} - 2, \text{rows} - 1)$. The 3x7 test in `main()` checks $C(8, 2) = 28$. - *Return one of the paths, not the count.* Keep a full 2-D table, then walk back from the goal: at each cell step to whichever of above/left has a non-zero count, and reverse the list. - *Could long long overflow too?* In principle, on a 100x100 grid with a huge walled-off region, yes. Two robust fixes: use `unsigned long long`, whose overflow is defined to wrap around modulo 2^{64} so the final answer (which fits) still comes out exact, or do a first pass that marks cells which cannot reach the goal and treat them as obstacles. - *Minimum-cost path instead of counting?* Same sweep with `cost[r][c] + min(above, left)`; that is the next problem.

8. Minimum Path Sum (LeetCode 64, medium)

The problem in plain words. You are given a grid of non-negative integers. Each number is the cost of stepping on that cell. Start at the top-left cell, finish at the bottom-right cell, and you may only move **right** or **down**. The cost of a path is the sum of every cell on it, including the first and the last. Return the smallest possible cost.

```

grid =  1  3  1
        1  5  1
        4  2  1

```

Cheapest path: 1 -> 3 -> 1 -> 1 -> 1, going right, right, down, down.
Cost = 1 + 3 + 1 + 1 + 1 = 7.

The path down the left side would be 1 + 1 + 4 + 2 + 1 = 9.
Going through the 5 costs at least 1 + 1 + 5 + 1 + 1 = 9.

Edge cases: a single cell returns that cell's value (`[[5]]` gives 5). A single row or a single column has only one path, so the answer is the sum of the row or column (`[[1],[2],[3]]` gives 6). The grid is at most 200 by 200 and every value is between 0 and 200, so an `int` is plenty. Because you can only go right or down, there is always at least one path; there is no "no path" case.

By hand. You could list every path and add them up, but there are far too many on a big grid. Instead notice that the last step onto any cell came either from the cell **above** it or from the cell to its **left**. So if you already know the cheapest way to reach those two cells, the cheapest way to reach this one is the cheaper of the two, plus this cell's own cost. That is the grid DP from A5, with `min` in place of the sum. Fill in a cost table from the top-left corner outward. The top row can only be reached from the left, the left column only from above.

```

grid:   1  3  1      cost:  1  4  5
        1  5  1          2  7  6
        4  2  1          6  8  7  <- answer, bottom-right

```

Reading a few cells: $\text{cost}[0][1] = 1 + 3 = 4$ (only from the left). $\text{cost}[1][0] = 1 + 1 = 2$ (only from above). $\text{cost}[1][1] = \min(\text{above } 4, \text{left } 2) + 5 = 7$. $\text{cost}[2][2] = \min(\text{above } 6, \text{left } 8) + 1 = 7$.

The idea. 1. $\text{cost}[r][c]$ means "cheapest total to arrive at (r, c) ". 2. Base cases: $\text{cost}[0][0] = \text{grid}[0][0]$; along the top row $\text{cost}[0][c] = \text{cost}[0][c-1] + \text{grid}[0][c]$; down the left column $\text{cost}[r][0] = \text{cost}[r-1][0] + \text{grid}[r][0]$. 3. Everywhere else: $\text{cost}[r][c] = \min(\text{cost}[r-1][c], \text{cost}[r][c-1]) + \text{grid}[r][c]$. 4. Fill row by row, left to right, so both the cell above and the cell to the left are already done. Return the bottom-right entry.

The Python you would have written.

```
def minPathSum(grid):
    rows, cols = len(grid), len(grid[0])
    cost = [[0] * cols for _ in range(rows)]
    for r in range(rows):
        for c in range(cols):
            if r == 0 and c == 0:
                cost[r][c] = grid[0][0]
            elif r == 0:
                cost[r][c] = cost[r][c - 1] + grid[r][c]      # only from the left
            elif c == 0:
                cost[r][c] = cost[r - 1][c] + grid[r][c]      # only from above
            else:
                cost[r][c] = min(cost[r - 1][c], cost[r][c - 1]) + grid[r][c]
    return cost[rows - 1][cols - 1]
```

In C++. The C++ version keeps only **one row** of the table instead of the whole thing. This is the "rolling row" trick from problem 7. Read it slowly the first time.

```
// 08_minimum_path_sum.cpp - LeetCode 64. Minimum Path Sum (medium) - DP
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

// dp[c] = cheapest cost to reach (current row, c). Rolling row again:
// before the update dp[c] is the cost from above, dp[c-1] the cost from the left.
int minPathSum(std::vector<std::vector<int>>& grid) {
    const int rows = static_cast<int>(grid.size());
    const int cols = rows ? static_cast<int>(grid[0].size()) : 0;
    if (rows == 0 || cols == 0) return 0;
    std::vector<int> dp(cols, 0);
    dp[0] = grid[0][0];
    for (int c = 1; c < cols; ++c) dp[c] = dp[c - 1] + grid[0][c]; // first row: only from the left
    for (int r = 1; r < rows; ++r) {
        dp[0] += grid[r][0]; // first column: only from above
        for (int c = 1; c < cols; ++c) dp[c] = std::min(dp[c], dp[c - 1]) + grid[r][c];
    }
    return dp[cols - 1];
}

int main() {
    {
        std::vector<std::vector<int>> g = {{1, 3, 1}, {1, 5, 1}, {4, 2, 1}};
        assert(minPathSum(g) == 7); // 1->3->1->1->1
    }
    {
        std::vector<std::vector<int>> g = {{1, 2, 3}, {4, 5, 6}};
        assert(minPathSum(g) == 12);
    }
    {
        std::vector<std::vector<int>> g = {{5}};
        assert(minPathSum(g) == 5);
    }
    {
        std::vector<std::vector<int>> g = {{1}, {2}, {3}};
    }
}
```

```

    assert(minPathSum(g) == 6); // single column
}
{
    std::vector<std::vector<int>> g = {{9, 1, 1}, {9, 9, 1}, {9, 9, 1}};
    assert(minPathSum(g) == 13); // hug the top and right edges
}
std::cout << "OK 08_minimum_path_sum.cpp\n";
return 0;
}

```

What is new:

- `std::vector<int> dp(cols, 0);` is a single row of `cols` zeros. Row `r` of the table only ever looks at row `r-1` and at itself, so we do not need to keep older rows. `dp` holds "the row we are currently filling in". The full-table version would be `std::vector<std::vector<int>> cost(rows, std::vector<int>(cols, 0));` exactly like the Python; both are correct, the rolling row just uses less memory.
- How the rolling row reads "above" and "left". When we are on row `r` and about to update `dp[c]`, we have not yet overwritten it, so `dp[c]` still holds the value from row `r-1`: that is "above". `dp[c-1]` has just been overwritten for row `r`: that is "left". So `std::min(dp[c], dp[c - 1]) + grid[r][c]` is exactly `min(above, left) + here`.
- `for (int c = 1; c < cols; ++c) dp[c] = dp[c - 1] + grid[0][c];` fills in the top row: a running sum, because the top row can only be reached from the left.
- `dp[0] += grid[r][0];` at the start of every later row is the left column: the old `dp[0]` is the cell above, so add this cell's cost. It replaces the Python `elif c == 0` branch.
- `std::min(a, b)` is Python's `min(a, b)` for two values. It lives in `<algorithm>`, hence the include at the top.
- `const int cols = rows ? static_cast<int>(grid[0].size()) : 0;` is the same empty-grid guard you saw in problem 4: do not touch `grid[0]` if there are no rows.
- `std::vector<std::vector<int>>& grid` takes the grid by reference to avoid copying 40,000 ints. The function never writes to it; LeetCode's signature just happens to be non-const.

Walk through the example. `dp` starts as the top row of costs: after the first loop, `dp = [1, 4, 5]`.

```

row 1:  dp[0] += 1          -> dp = [2, 4, 5]      (4 and 5 are still "above")
        c=1: min(4, 2) + 5 = 7 -> dp = [2, 7, 5]
        c=2: min(5, 7) + 1 = 6 -> dp = [2, 7, 6]      this is row 1 of the cost table
row 2:  dp[0] += 4          -> dp = [6, 7, 6]
        c=1: min(7, 6) + 2 = 8 -> dp = [6, 8, 6]
        c=2: min(6, 8) + 1 = 7 -> dp = [6, 8, 7]      this is row 2 of the cost table
return dp[2] = 7

```

Compare with the full table drawn above: each `dp` after a row is exactly that row.

Complexity. Every cell is visited once with constant work, so time is proportional to `rows × cols`, $O(\text{rows} \cdot \text{cols})$. Memory is one row, $O(\text{cols})$; the Python version uses $O(\text{rows} \cdot \text{cols})$.

Robotics. This is a cost-to-go table over a traversability map: each cell stores the cheapest way to reach it, and a robot can then follow the table by stepping to the cheaper neighbour. The right-or-down restriction is what makes a plain table fill work. Allow all four directions and the table has cycles, so you need Dijkstra instead; that is what the NavFn global planner in ROS computes over the whole costmap before it extracts a path by walking downhill.

Follow-ups you may get. - What if you can move in all four directions? Then a cell can depend on cells not yet filled, so row-by-row DP is wrong. Use Dijkstra (A without the heuristic, Part C) with cell cost as the edge weight. - Negative cell values? The DP still works because right/down paths never revisit a cell. Dijkstra would not. - Return the path, not just the cost? Keep the full table, then walk back from the bottom-right: at each cell step to whichever of above/left has cost equal to `cost[r][c] - grid[r][c]`.

9. Word Search (LeetCode 79, medium)

The problem in plain words. You are given a board of letters and a word. Return `true` if the word can be spelled by walking on the board: start at any cell, and each next letter must be in a cell directly **next to** the previous one (up, down, left or right; not diagonal). You may **not** step on the same cell twice within one spelling. Otherwise return `false`.

```
board =  A  B  C  E
         S  F  C  S
         A  D  E  E

word = "ABCCED"  -> true    A(0,0) B(0,1) C(0,2) C(1,2) E(2,2) D(2,1)
word = "SEE"     -> true    S(1,3) E(2,3) E(2,2)
word = "ABCB"    -> false   A(0,0) B(0,1) C(0,2) and then B again would have to
                           be the same B at (0,1), which is not allowed
```

Edge cases: a 1 by 1 board with 'a' and word "a" is true; the same board with "ab" is false. An empty board or empty word gives false. The board is at most 6 by 6 and the word at most 15 letters, which is a strong hint that trying every path is acceptable.

By hand. Put your finger on every cell that holds the first letter. From there, look at the four neighbours for the second letter, move your finger, and keep going. Two things happen. When no neighbour has the next letter you are stuck: lift your finger, go back one cell, and try a different neighbour there. And when you consider a neighbour, you must ignore cells your finger has already passed over on *this* attempt. That is backtracking from A4: mark the cell as in use, try the neighbours, unmark it when you come back.

The neat trick in this solution is that the board itself is the "in use" list. When you step on a cell you overwrite its letter with '#'. No letter in the word is ever '#', so a marked cell can never match, which is exactly what "may not reuse" means. When you step back off the cell, you write the original letter back.

The idea. 1. `dfs(r, c, i)` answers: "standing on (r, c), can I match `word[i:]` from here?" 2. If `i` equals the word length, every letter has been matched: return `true`. 3. If (r, c) is outside the board, or `board[r][c]` is not `word[i]`, return `false`. A '#' cell fails this test automatically. 4. Otherwise save the letter, write '#', and ask the four neighbours about `i + 1`. If any says `true`, the answer is `true`. 5. Put the saved letter back **before** returning, whatever the answer was. 6. `exist` calls `dfs(r, c, 0)` from every cell and returns `true` the first time it succeeds.

The Python you would have written. The board must be a list of lists (not a list of strings) so that cells can be overwritten.

```
def exist(board, word):
    rows, cols = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word):
            return True
        if not (0 <= r < rows and 0 <= c < cols) or board[r][c] != word[i]:
            return False
        saved = board[r][c]
        board[r][c] = '#' # mark: '#' never matches a letter
        found = (dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1) or
                dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1))
        board[r][c] = saved # unmark
        return found
    return any(dfs(r, c, 0) for r in range(rows) for c in range(cols))
```

In C++.

```
// 09_word_search.cpp - LeetCode 79. Word Search (medium) - backtracking
#include <cassert>
#include <iostream>
#include <string>
#include <vector>
```

```

// Backtracking: mark the cell as used, recurse on the next letter, unmark on the
// way out so a different path can reuse it. Unmarking is what makes this
// backtracking rather than plain DFS.
bool dfs(std::vector<std::vector<char>>& board, const std::string& word, int r, int c, int i) {
    const int rows = static_cast<int>(board.size());
    const int cols = static_cast<int>(board[0].size());
    if (i == static_cast<int>(word.size())) return true;
    if (r < 0 || r >= rows || c < 0 || c >= cols || board[r][c] != word[i]) return false;
    const char saved = board[r][c];
    board[r][c] = '#'; // in-place visited mark; '#' never matches a letter
    const bool found = dfs(board, word, r + 1, c, i + 1) || dfs(board, word, r - 1, c, i + 1) ||
        dfs(board, word, r, c + 1, i + 1) || dfs(board, word, r, c - 1, i + 1);
    board[r][c] = saved; // unmark: the board is restored for the next start cell
    return found;
}

bool exist(std::vector<std::vector<char>>& board, std::string word) {
    if (board.empty() || board[0].empty() || word.empty()) return false;
    const int rows = static_cast<int>(board.size());
    const int cols = static_cast<int>(board[0].size());
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c)
            if (dfs(board, word, r, c, 0)) return true;
    return false;
}

int main() {
    std::vector<std::vector<char>> board = {
        {'A', 'B', 'C', 'E'},
        {'S', 'F', 'C', 'S'},
        {'A', 'D', 'E', 'E'};
    assert(exist(board, "ABCCED") == true);
    assert(exist(board, "SEE") == true);
    assert(exist(board, "ABCB") == false); // would need to reuse the B
    // Board is restored after every search.
    assert(board[0][0] == 'A' && board[1][1] == 'F');

    std::vector<std::vector<char>> one = {{'a'}};
    assert(exist(one, "a") == true);
    assert(exist(one, "ab") == false);
    std::vector<std::vector<char>> empty;
    assert(exist(empty, "a") == false);

    // Backtracking matters: seven A's ring the B. Starting at (0,0) the DFS
    // walks a short arc, fails to find C, unmarks, and a later start at (1,2)
    // snakes the long way round: (1,2)(0,2)(0,1)(0,0)(1,0)(2,0)(2,1) then C.
    std::vector<std::vector<char>> tricky = {
        {'A', 'A', 'A'},
        {'A', 'B', 'A'},
        {'A', 'A', 'C'};
    assert(exist(tricky, "AAAAAAAC") == true); // 7 A's then C
    assert(exist(tricky, "AAAAAAAC") == false); // 8 A's: one too many
    assert(exist(tricky, "ABA") == true);
    assert(exist(tricky, "ABC") == false); // B and C are only diagonal neighbours
    std::cout << "OK 09_word_search.cpp\n";
    return 0;
}

```

What is new:

- `dfs` is a free function above `exist` (no nested functions in C++), so the board and the word are passed in. `std::vector<std::vector<char>>& board` is a reference: the marks written by one call must be visible to every other call, and the unmark must land on the same board. `const std::string& word` is a reference too, to avoid copying the word on every call; `const` because we only read it.
- `word[i]` is a `char` and `board[r][c]` is a `char`, so `board[r][c] != word[i]` compares two single characters, the same as Python's `!=` on one-letter strings.

- `if (i == static_cast<int>(word.size())) return true;` is `if i == len(word)`. The cast is the signed/unsigned issue from A1 again.
- `const char saved = board[r][c]; board[r][c] = '#';` is the mark. A `char` literal uses single quotes in C++; `"#"` with double quotes would be a string and would not compile here.
- The four recursive calls joined by `||` are Python's `or` chain. C++ `||` also short-circuits: as soon as one call returns `true`, the remaining calls are skipped.
- `const bool found = ...; board[r][c] = saved; return found;` is the important shape. The result is stored in a variable so the `unmark` **always** runs before the function returns. If you wrote `if (dfs(...)) return true;` inside the chain, the `'#'` would stay on the board after a success and the caller's board would be corrupted. The test `assert(board[0][0] == 'A' && board[1][1] == 'F');` in `main` checks exactly this.
- `bool exist(std::vector<std::vector<char>>& board, std::string word)` takes `word` by value, which is fine for a 15-letter string and matches LeetCode's signature.

Walk through the example. The code tries neighbours in the order down, up, right, left.

First, an attempt that fails: "ABCB" starting at (0,0). A matches, mark it. Need B: down is S, no; up is outside; right (0,1) is B, mark it. Need C: down F, no; up outside; right (0,2) is C, mark it. The board now looks like this:

```
# # # E
S F C S
A D E E
```

Need B from (0,2): down (1,2) is C, no; up outside; right (0,3) is E, no; left (0,1) is '#', no. The only B is under a mark, so this attempt is dead. `dfs` at (0,2) writes C back and returns `false`. Back at (0,1) the last direction, left, is (0,0) which is '#', no; write B back, return `false`. Back at (0,0), left is outside; write A back, return `false`. The board is exactly as it was:

```
A B C E
S F C S
A D E E
```

`exist` keeps scanning. The other A at (2,0) has neighbours S and D, neither is B, so it fails immediately. No start works: `false`.

Now the attempt that succeeds: "ABCCED" starting at (0,0). Same first three steps as before, so (0,0), (0,1), (0,2) get marked. Need C from (0,2): down (1,2) is C, mark it. Need E: down (2,2) is E, mark it. Need D from (2,2): down is outside; up (1,2) is '#', no; right (2,3) is E, no; left (2,1) is D, mark it. Now `i` is 6, which equals the word length, so the next call returns `true` straight away.

```
# # # E      every '#' is a letter of ABCCED, in use on this path
S F # S
A # # E
```

`true` travels back up the chain. Each level still runs its `unmark` line on the way out, so by the time `exist` returns `true` the board is fully restored.

Complexity. There are `rows × cols` starting cells. From each, the first step has up to 4 choices and every later step at most 3 (you never go back to the marked cell you came from), so the worst case is proportional to `rows × cols × 3L` where `L` is the word length: $O(M \cdot N \cdot 3^L)$. With a 6 by 6 board and 15 letters this is fine. The recursion is at most `L` deep, so memory is $O(L)$.

Robotics. Backtracking with state restoration is the skeleton of any small exhaustive search: enumerating coverage paths on a small grid, trying task orderings for a robot arm, or assigning robots to goals under constraints. In each case you commit to a choice, recurse, and undo the choice so the next candidate starts from the same state, exactly like writing `'#'` and then writing the letter back.

Follow-ups you may get. - *Many words on one board?* That is Word Search II (LeetCode 212): put all the words in a trie and run the same DFS once, stopping as soon as the current prefix is not in the trie. - *Can you prune before searching?* Count the letters on the board; if the word needs more of any letter than the board has, return `false` without a single DFS call. - *Why not a separate `visited` table?* You can, and you must if the board is `const`. The `'#'` trick is shorter and uses no extra memory; just remember to restore.

10. Pacific Atlantic Water Flow (LeetCode 417, medium)

The problem in plain words. This one is famous for being hard to read, so here it is slowly. You are given a grid of heights; think of it as a rectangular island. The **Pacific Ocean** touches the island along its **top edge and left edge**. The **Atlantic Ocean** touches it along the **bottom edge and right edge**.

		P a c i f i c							
		+-----+							
			1	2	2	3	5		
P			3	2	3	4	4		A
a			2	4	5	3	1		t
c			6	7	1	4	5		l
			5	1	1	2	4		a
		+-----+							n
		A t l a n t i c							t i c

Rain falls on a cell. Water can flow from a cell to any of its four neighbours (up, down, left, right) whose height is **equal or lower**. It can keep flowing like that, cell to cell, as long as it never goes uphill. Water that flows off an edge of the grid falls into the ocean on that edge. Since every cell in the top row and left column is already touching the Pacific, rain on those cells reaches the Pacific with no flowing at all; likewise the bottom row and right column for the Atlantic.

The question: return the list of cells `[r, c]` such that rain on that cell can reach **both** oceans (by two different routes if necessary). Any order is accepted by LeetCode; our code returns them row by row.

For the grid above the answer is 7 cells, marked *:

1	2	2	3	[5]*	(0,4): top edge (Pacific) and right edge (Atlantic) at once
3	2	3	[4]*[4]*		(1,3): 4 -> 3 -> 2 -> 2 -> 1 up to the Pacific, 4 -> 4 right to the Atlantic
2	4	[5]*	3	1	(2,2): 5 -> 4 -> 2 left to the Pacific, 5 -> 3 -> 1 right to the Atlantic
[6]*	[7]*	1	4	5	(3,0): left edge, and 6 -> 5 down to the bottom edge
[5]*	1	1	2	4	(4,0): left edge (Pacific) and bottom edge (Atlantic) at once

Edge cases: a 1 by 1 grid touches all four edges, so the answer is `[[0,0]]`. An empty grid gives an empty list. A completely flat grid returns every cell (water can flow anywhere on a flat surface). A cell that is lower than all four neighbours, like the 0 in the middle of a 3 by 3 ring of 1s, cannot flow anywhere, so it is in the answer only if it sits on an edge. The grid is at most 200 by 200 and heights are between 0 and 100,000.

By hand. The obvious approach is to pick each cell, follow the water downhill in every possible direction, and see which oceans you hit. On the 5 by 5 example that is 25 separate searches, and on 200 by 200 it is 40,000 searches over 40,000 cells each: far too slow.

The trick is to turn the question around. Instead of asking "can this cell's water get down to the Pacific?", ask "starting from the Pacific, which cells can I **climb up** to?" Water flows from higher to equal-or-lower, so if you walk in reverse you go from lower to equal-or-higher. Start with every cell on the Pacific edge (they can obviously reach the Pacific), and spread to any neighbour whose height is **at least** the current cell's height. Every cell you reach this way can send water down the same route in reverse. That is one multi-source BFS from A2, with the visited rule changed from "not a wall" to "not lower than me". Do the same from the Atlantic edge. The answer is the cells reached by both.

The idea. 1. Make two boolean tables `pac` and `atl`, all `false`. 2. Push every cell of row 0 and column 0 into a Pacific queue and mark it in `pac`. Push every cell of the last row and last column into an Atlantic queue and mark it in `atl`. 3. Run BFS on each queue. A neighbour is allowed if it is inside the grid, not yet marked in this table, and `heights[neighbour] >= heights[current]`. Mark it and push it. 4. Return every `(r, c)` with `pac[r][c]` and `atl[r][c]` both true.

The Python you would have written.

```
from collections import deque

def pacificAtlantic(heights):
    rows, cols = len(heights), len(heights[0])
    def climb(starts):
        reach = [[False] * cols for _ in range(rows)]
        q = deque()
        for r, c in starts:
            reach[r][c] = True
            q.append((r, c))
        while q:
            r, c = q.popleft()
            for dr, dc in ((-1, 0), (0, 1), (1, 0), (0, -1)):
                nr, nc = r + dr, c + dc
                if (0 <= nr < rows and 0 <= nc < cols and not reach[nr][nc]
                    and heights[nr][nc] >= heights[r][c]): # uphill or flat only
                    reach[nr][nc] = True
                    q.append((nr, nc))
        return reach
    pac_edge = {(r, 0) for r in range(rows)} | {(0, c) for c in range(cols)}
    atl_edge = {(r, cols - 1) for r in range(rows)} | {(rows - 1, c) for c in range(cols)}
    pac = climb(pac_edge)
    atl = climb(atl_edge)
    return [[r, c] for r in range(rows) for c in range(cols) if pac[r][c] and atl[r][c]]
```

The edges are sets so that the corner cell `(0, 0)`, which is in both row 0 and column 0, is not pushed twice.

In C++.

```
// 10_pacific_atlantic.cpp - LeetCode 417. Pacific Atlantic Water Flow (medium)
// Reverse BFS from each ocean: instead of asking "can this cell reach the
// ocean?" (n*m searches), ask "which cells can the ocean climb up to?" (2 searches).
#include <cassert>
#include <iostream>
#include <queue>
#include <utility>
#include <vector>

// Multi-source BFS that moves only to neighbours with height >= current.
void climb(const std::vector<std::vector<int>>& h, std::vector<std::vector<char>>& reach,
    std::queue<std::pair<int, int>>& q) {
    const int rows = static_cast<int>(h.size());
    const int cols = static_cast<int>(h[0].size());
    constexpr int dr[4] = {-1, 0, 1, 0};
    constexpr int dc[4] = {0, 1, 0, -1};
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        for (int k = 0; k < 4; ++k) {
            const int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
            if (reach[nr][nc] || h[nr][nc] < h[r][c]) continue; // water flows downhill, so we climb uphill
            reach[nr][nc] = 1;
            q.push({nr, nc});
        }
    }
}

std::vector<std::vector<int>> pacificAtlantic(std::vector<std::vector<int>>& heights) {
    std::vector<std::vector<int>> out;
    const int rows = static_cast<int>(heights.size());
    const int cols = rows ? static_cast<int>(heights[0].size()) : 0;
```

```

if (rows == 0 || cols == 0) return out;
// vector<char> instead of vector<bool>: bool is a bit-packed proxy and you cannot take references into it.
std::vector<std::vector<char>> pac(rows, std::vector<char>(cols, 0));
std::vector<std::vector<char>> atl(rows, std::vector<char>(cols, 0));
std::queue<std::pair<int, int>> qp, qa;
for (int r = 0; r < rows; ++r) {
    pac[r][0] = 1; qp.push({r, 0});
    atl[r][cols - 1] = 1; qa.push({r, cols - 1});
}
for (int c = 0; c < cols; ++c) {
    if (!pac[0][c]) { pac[0][c] = 1; qp.push({0, c}); }
    if (!atl[rows - 1][c]) { atl[rows - 1][c] = 1; qa.push({rows - 1, c}); }
}
climb(heights, pac, qp);
climb(heights, atl, qa);
for (int r = 0; r < rows; ++r)
    for (int c = 0; c < cols; ++c)
        if (pac[r][c] && atl[r][c]) out.push_back({r, c});
return out; // row-major order, which is what the tests below expect
}

int main() {
    {
        std::vector<std::vector<int>> h = {
            {1, 2, 2, 3, 5},
            {3, 2, 3, 4, 4},
            {2, 4, 5, 3, 1},
            {6, 7, 1, 4, 5},
            {5, 1, 1, 2, 4}};
        const std::vector<std::vector<int>> want = {{0, 4}, {1, 3}, {1, 4}, {2, 2}, {3, 0}, {3, 1}, {4, 0}};
        assert(pacificAtlantic(h) == want);
    }
    {
        std::vector<std::vector<int>> h = {{1}};
        const std::vector<std::vector<int>> want = {{0, 0}}; // touches both oceans
        assert(pacificAtlantic(h) == want);
    }
    {
        std::vector<std::vector<int>> h;
        assert(pacificAtlantic(h).empty());
    }
    {
        // Flat grid: every cell reaches both oceans.
        std::vector<std::vector<int>> h = {{1, 1}, {1, 1}};
        assert(pacificAtlantic(h).size() == 4);
    }
    {
        // A pit: the centre 0 cannot flow uphill to any neighbour, the 8 border cells can.
        std::vector<std::vector<int>> h = {{1, 1, 1}, {1, 0, 1}, {1, 1, 1}};
        const auto out = pacificAtlantic(h);
        assert(out.size() == 8);
        for (const auto& rc : out) assert(!(rc[0] == 1 && rc[1] == 1));
    }
    {
        // (0,0) is the lowest cell: it touches the Pacific but nothing lets it climb to the Atlantic side.
        std::vector<std::vector<int>> h = {{1, 2}, {3, 4}};
        const std::vector<std::vector<int>> want = {{0, 1}, {1, 0}, {1, 1}};
        assert(pacificAtlantic(h) == want);
    }
    std::cout << "OK 10_pacific_atlantic.cpp\n";
    return 0;
}

```

What is new:

- `climb` is the BFS from A2 with one line changed: `if (reach[nr][nc] || h[nr][nc] < h[r][c]) continue;` skips neighbours that are already reached **or lower** than the current cell. Remember the direction is flipped because we search uphill.
- `climb` takes the queue **by reference** (`std::queue<std::pair<int,int>>& q`) and the reach table by reference. The caller fills the queue with the edge cells, then hands it over. This lets one function serve both oceans. `h` is a `const` reference because it is only read.

- `std::vector<std::vector<char>>` for the two reach tables instead of `std::vector<bool>`. In C++ `std::vector<bool>` is a special case that packs eight flags into a byte, and because of that you cannot take an ordinary reference to one of its elements. Using `char` with the values 0 and 1 behaves like a normal container and is the usual workaround. `if (pac[r][c] && atl[r][c])` works because a non-zero `char` counts as true.
- `constexpr int dr[4] = ...` is the same as `const int dr[4]` from A1; `constexpr` just promises the value is known at compile time. Either spelling is fine.
- `std::queue<std::pair<int, int>> qp, qa;` declares two queues in one statement.
- `if (!pac[0][c]) { pac[0][c] = 1; qp.push({0, c}); }` is the "do not push the corner twice" check, the same job the Python sets do.
- `out.push_back({r, c});` appends a two-element `std::vector<int>` built from the braces, so the result has the shape `[[r, c], ...]` that LeetCode expects.
- `std::vector<std::vector<int>> out;` is declared first and returned in the early-exit case, so an empty grid returns an empty list.

Walk through the example. The Pacific queue starts with the nine edge cells: (0,0) (1,0) (2,0) (3,0) (4,0) (0,1) (0,2) (0,3) (0,4), all marked P.

```
pop (0,0) h=1: right (0,1) and down (1,0) already marked.
pop (1,0) h=3: right (1,1) h=2 is lower, skip.
pop (2,0) h=2: right (2,1) h=4 >= 2, mark, push.
pop (3,0) h=6: right (3,1) h=7 >= 6, mark, push.
pop (4,0) h=5: right (4,1) h=1, skip.
pop (0,1) h=2: down (1,1) h=2 >= 2 (equal is allowed), mark, push.
pop (0,2) h=2: down (1,2) h=3, push.    pop (0,3) h=3: down (1,3) h=4, push.
pop (0,4) h=5: down (1,4) h=4, skip.
pop (2,1) h=4: right (2,2) h=5, push.    pop (3,1) h=7: neighbours 1 and 1, skip.
pop (1,1), (1,2): nothing new.          pop (1,3) h=4: right (1,4) h=4, push.
pop (2,2) h=5: right 3 and down 1, skip. pop (1,4) h=4: down (2,4) h=1, skip. Queue empty.
```

The Atlantic search starts from the last row and last column and climbs the same way. The two reach tables, and their overlap:

pac:	P P P P P	atl:	 A	both:	 *
	P P P P P		. . . A A		. . . * *
	P P P . .		. . A A A		. . * . .
	P P . . .		A A A A A		* * . . .
	P		A A A A A		*

Reading both row by row gives (0,4) (1,3) (1,4) (2,2) (3,0) (3,1) (4,0), the seven cells from the drawing at the top.

Complexity. Two BFS passes, each marking every cell at most once: time proportional to rows × cols, $O(\text{rows} \cdot \text{cols})$. Memory is the two reach tables and the queues, also $O(\text{rows} \cdot \text{cols})$. The naive "search from every cell" approach would be $O((\text{rows} \cdot \text{cols})^2)$.

Robotics. Searching backwards from the goal set is standard practice: a wavefront run from all docking stations at once tells every cell which dock is nearest, in one pass instead of one per cell. The "only climb to equal-or-higher" rule is a traversability constraint, like a slope limit on a terrain map, and intersecting two reachability sets is how you find cells a robot can reach from its current position *and* from which it can still reach a rendezvous point.

Follow-ups you may get. - *Why not DFS from every cell?* It is correct but $O((\text{rows} \cdot \text{cols})^2)$. Two searches from the oceans give the same answer in linear time. - *Only the count is needed?* Same code; count the cells where both tables are set instead of building the output vector. - *Water can only flow strictly downhill (not to equal heights)?* Change `<` to `<=` in the skip test in `climb`; the structure is unchanged.

Part C — Build: A* on a grid, from scratch, with tests

Everything in Part B was one function with a `main()` under it. This part is different: you build a small *project*. A planner library, a demo program that draws a path on a map, and a test suite that proves the planner is right. It is the kind of thing you can put in a public repository and talk about, and it is the algorithm that the global planner of a ROS navigation stack actually runs.

Rough time: two days. Day 1 is C1 to C3 (understand A* and write the 40-line version yourself). Day 2 is C4 and C5 (read the project, build it, break it).

You need A2 (the BFS ripple) cold before starting. A* is BFS with a smarter queue; if the queue version is not automatic yet, go back and type it out once more.

C1. What A* is, with a tiny example

Here is the map from A1 again, now with a start `S` and a goal `G`. `#` is a wall. Moves are up, down, left, right, and every move costs 1. The question: what is the cheapest way from `S` to `G`?

	c0	c1	c2	c3
r0	S	.	.	.
r1	.	#	#	.
r2	.	.	.	G

By eye: along the top row and down the right side is 5 moves. Down the left side and along the bottom row is also 5. There is nothing cheaper, because `G` is 2 rows down and 3 columns right, and each move changes the row or the column by one, so you need at least $2 + 3 = 5$ moves.

BFS from A2 answers this. It also fills the *whole* map with distances before it is done, including the bottom-left corner, which is nowhere near the goal. That is the waste A* removes.

Three numbers per cell. A* keeps, for every cell it has touched:

- `g`: the cost of the cheapest way we have found *so far* from `S` to this cell. Exact, because we added up real moves.
- `h`: a *guess* of the cost still to go from this cell to `G`. On a 4-direction grid the natural guess is "rows still to go plus columns still to go", the Manhattan distance. For `(0,1)` that is $|0 - 2| + |1 - 3| = 4$.
- `f = g + h`: the guessed total cost of a route through this cell.

The to-do list. Instead of BFS's queue (oldest first), A keeps a *to-do list of cells it has seen but not yet dealt with, and always takes the cell with the smallest f**: the one that looks like it lies on the cheapest total route. Taking a cell means: look at its neighbours, and for each one work out `g` (my `g` plus 1), `h`, `f`, and put it on the list. When the cell taken from the list is `G`, stop; its `g` is the answer.

Here is every step on the map above. "g/h/f" is written per cell.

step	to-do list (cell: g/h/f)	taken	added to the list
0	(0,0): 0/5/5	(0,0)	(0,1): 1/4/5 and (1,0): 1/4/5
1	(0,1): 1/4/5, (1,0): 1/4/5	(0,1). A tie: same f, same g. We take the one with the smaller row number; any fixed rule will do.	(0,2): 2/3/5. (1,1) is a wall.
2	(0,2): 2/3/5, (1,0): 1/4/5	(0,2). Same f again; among equal f we prefer the larger g (more on this below).	(0,3): 3/2/5. (1,2) is a wall.
3	(0,3): 3/2/5, (1,0): 1/4/5	(0,3)	(1,3): 4/1/5

step	to-do list (cell: g/h/f)	taken	added to the list
4	(1,3): 4/1/5, (1,0): 1/4/5	(1,3)	(2,3): 5/0/5
5	(2,3): 5/0/5, (1,0): 1/4/5	(2,3). This is the goal. Answer: g = 5.	stop

Mark the cells that were taken off the list with x:

```

      c0  c1  c2  c3
r0    S   x   x   x
r1    .   #   #   x
r2    .   .   .   G

```

Six cells taken. (1,0) sat on the list the whole time and was never taken, and the three bottom-left cells were never even added. BFS on the same map takes all 10 free cells before it reaches G. That is the whole point: the guess *h* pulls the search toward the goal.

One thing to notice in the table: every *f* was 5. That is because the guess happened to be exactly right along the top row (no detour was needed). When a detour *is* needed, cells on the detour get a larger *f*, and A* only takes them once every cell with a smaller *f* has been tried. *f* never has to go down, because *h* is only ever an under-estimate (rule 2 below).

A bigger picture: BFS versus A* on an open 5x5. Start top-left, goal bottom-right, no walls. x marks a cell that was taken off the list before the goal was reached.

BFS (25 cells taken)	A* (9 cells taken)
S x x x x	S x x x x
x x x x x	 x
x x x x x	 x
x x x x x	 x
x x x x G	 G

BFS grows its ripple in every direction, and the goal is the very last cell it reaches, so it looks at everything. A* walks straight there.

A detail hidden in that picture: on an open grid every cell has *f* = 8 (its *g* plus its *h* always adds up to the true distance, because nothing is in the way). So *which* *f* = 8 cell to take is decided purely by the tie-break. The rule used here, and in the project below, is: among equal *f*, take the cell with the *larger g*, the one that has already travelled furthest. With the opposite rule (smaller *g* first) A* on this open grid takes all 25 cells, exactly like BFS. The tie-break is not a detail you can skip.

The three rules.

- h = 0 turns A* into Dijkstra.** With no guess at all, *f* = *g*, and "take the smallest *f*" becomes "take the cheapest cell found so far". That is Dijkstra's algorithm. On a grid where every move costs 1 it visits cells in the same order as BFS, just with a slower queue. So *A is Dijkstra plus a guess, and Dijkstra is A* with a guess of zero. The project uses this: `dijkstra()` is literally `aStar()` with the zero heuristic.
- h must never over-estimate.** A guess that is too small only makes A *look at a few more cells*. A guess that is too big can make it return the wrong answer. A guess that never over-estimates is called *admissible**. Here is a bad one, on a map where the two routes have different lengths:

```
c0 c1 c2 c3 r0 S . . . r1 . # # . r2 . # # G r3 . . . .
```

The top route costs 5, the bottom route costs 7. Now suppose the guess for cell (0,3) is wrong and says 20 (the honest value is 2). Then (0,3) gets *f* = 3 + 20 = 23 and sinks to the bottom of the list. Every cell on the bottom route has *f* at most 7, so all of them are taken first, and the goal is taken with *g* = 7 from the

bottom. A returns 7. The real answer is 5. The over-estimate hid the best route. Manhattan on a 4-direction grid can never over-estimate (you cannot get anywhere in fewer than rows + cols moves), so it is safe. C3 shows a case where it is not* safe.

The heuristics used in this chapter also satisfy a slightly stronger condition called *consistent*: the guess never drops by more than the cost of one move. That is what allows us to mark a cell as finished the first time it is taken off the list and never look at it again. C4 points out the exact line that depends on it.

1. **The to-do list is a priority queue.** "Give me the entry with the smallest f" must be fast, and the list can hold thousands of entries. A plain list scanned every time would be proportional to its length per step. A binary heap does it in about $\log(n)$ steps. In Python that is `heapq`; in C++ it is `std::priority_queue`. Both push and pop in $\log(n)$.

In your own words: "A* is BFS with a to-do list ordered by $f = g + h$, where g is the exact cost so far and h is a guess of the cost to go. It always takes the smallest f . If h never over-estimates, the first time the goal is taken its g is the true shortest cost. With $h = 0$ it is Dijkstra."

C2. From BFS to A*: what changes in the code

Put the BFS loop from A2 next to the A* loop. Lines marked `<--` are the ones that change.

BFS (A2)	A*
-----	-----
<code>dist[start] = 0</code>	<code>g[start] = 0</code>
<code>queue = [start]</code>	<code>open = [(f = h(start), start)]</code>
<code>while queue not empty:</code>	<code>while open not empty:</code>
<code>cell = queue.pop_front()</code>	<code>cell = open.pop_smallest_f()</code> <-- cheapest-looking first, not oldest
	if cell is closed: continue <-- a stale duplicate; skip it
	mark cell closed
	if cell == goal: <-- stop at the goal
	rebuild path, return g[cell]
for each free neighbour n of cell:	for each free neighbour n of cell:
if dist[n] is unset:	new_g = g[cell] + cost(cell, n) <-- moves may cost different amounts
dist[n] = dist[cell] + 1	if g[n] is unset or new_g < g[n]: <-- "unset" became "cheaper than before"
	g[n] = new_g
	parent[n] = cell <-- remember how we got here
queue.push_back(n)	open.push((new_g + h(n), n))

Four changes: the queue is ordered by f ; a cell can be improved after it has been pushed, so we push again and skip stale copies later; we stop when the goal is taken, not when the queue runs dry; and we remember each cell's parent so the path can be rebuilt.

Now the same thing in real C++, on a 4-direction grid with every move costing 1. This is a complete file. It is saved as `code/03-grids-bfs-dfs/11_astar_minimal.cpp`; type it out yourself first, then compile with `g++ -std=c++17 -Wall -Wextra -O2 11_astar_minimal.cpp -o astar_min`, and run it. It prints the path through the tiny map and OK.

```
#include <algorithm>    // std::reverse
#include <cassert>
#include <cstdlib>      // std::abs
#include <functional>   // std::greater
#include <iostream>
#include <queue>        // std::priority_queue
#include <string>
#include <tuple>
#include <utility>      // std::pair
#include <vector>

// A* on a 4-connected grid ('#' = wall). Every step costs 1.
// Returns the cost of the cheapest path from (sr,sc) to (gr,gc), or -1 if there
// is none, and fills `path` with the cells from start to goal inclusive.
int aStar4(const std::vector<std::string>& grid, int sr, int sc, int gr, int gc,
          std::vector<std::pair<int, int>>& path) {
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    const int dr[4] = {-1, 0, 1, 0};
```

```

const int dc[4] = {0, 1, 0, -1};
auto h = [&](int r, int c) { return std::abs(r - gr) + std::abs(c - gc); }; // rows+cols to goal

std::vector<int> g(rows * cols, -1); // best cost so far per cell; -1 = never reached
std::vector<int> parent(rows * cols, -1); // which cell we came from
std::vector<char> closed(rows * cols, 0); // 1 once a cell has been taken off the list for good
using Entry = std::tuple<int, int, int, int>; // (f, -g, r, c): smallest f first, then LARGEST g
std::priority_queue<Entry, std::vector<Entry>, std::greater<Entry>> open;

g[sr * cols + sc] = 0;
open.push({h(sr, sc), 0, sr, sc});
while (!open.empty()) {
    const auto [f, negG, r, c] = open.top(); // the entry with the smallest f
    open.pop();
    const int idx = r * cols + c;
    if (closed[idx]) continue; // stale duplicate of a cell already done: skip it
    closed[idx] = 1;
    if (r == gr && c == gc) { // goal taken off the list: its g is the answer
        for (int i = idx; i != -1; i = parent[i]) path.push_back({i / cols, i % cols});
        std::reverse(path.begin(), path.end()); // we walked goal -> start, flip it
        return -negG;
    }
    for (int k = 0; k < 4; ++k) {
        const int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue; // outside
        if (grid[nr][nc] == '#') continue; // wall
        const int ni = nr * cols + nc;
        const int ng = -negG + 1; // one more step
        if (g[ni] == -1 || ng < g[ni]) { // first time here, or a cheaper way than before
            g[ni] = ng;
            parent[ni] = idx;
            open.push({ng + h(nr, nc), -ng, nr, nc}); // push even if ni is already in the list
        }
    }
}
return -1; // list ran empty: the goal is unreachable
}

int main() {
    const std::vector<std::string> tiny = {
        "S...",
        ".##.",
        "...G"};
    std::vector<std::pair<int, int>> path;
    assert(aStar4(tiny, 0, 0, 2, 3, path) == 5);
    assert(path.size() == 6);
    assert(path.front() == std::make_pair(0, 0));
    assert(path.back() == std::make_pair(2, 3));
    for (const auto& [r, c] : path) std::cout << "(" << r << "," << c << ") ";
    std::cout << "\n";

    const std::vector<std::string> walled = {
        "S#.",
        ".#.",
        ".#G"};
    path.clear();
    assert(aStar4(walled, 0, 0, 2, 2, path) == -1);
    assert(path.empty());

    path.clear();
    assert(aStar4(tiny, 0, 0, 0, 0, path) == 0 && path.size() == 1); // start == goal
    std::cout << "OK 11_astar_minimal.cpp\n";
    return 0;
}

```

Output:

```

(0,0) (0,1) (0,2) (0,3) (1,3) (2,3)
OK 11_astar_minimal.cpp

```

That is the top route from the C1 trace, cost 5.

What is new, line by line, for a Python developer.

- **Flat indexing.** `g`, `parent` and `closed` are one-dimensional vectors of size `rows * cols`, and `cell (r, c)` lives at `r * cols + c`. Going back is `i / cols` for the row and `i % cols` for the column. One allocation instead of `rows` of them, and it is how real occupancy grids are stored (A6). You will see it again in the project.
- `auto h = [&](int r, int c) { ... };` is a *lambda*: a small local function, like a nested `def` in Python. The `[&]` means "you may use the variables around you", here `gr` and `gc`. Calling it is just `h(r, c)`.
- `std::priority_queue<Entry, std::vector<Entry>, std::greater<Entry>> open;` is the heap. Three things in the angle brackets: what it holds, which container it is built on (always `std::vector`), and how to compare two entries. Here is the trap: by default `std::priority_queue` puts the *largest* element on top. That is the opposite of Python's `heapq`, which always pops the smallest. To get the smallest `f` on top you must hand it the "backwards" comparison `std::greater<Entry>`. If you forget, the search takes the most-expensive-looking cell every time and behaves like a search for the *longest* route. The project's test suite catches that mistake instantly; your own eyes may not.
- **The entry is a tuple (f, -g, r, c).** C++ tuples compare element by element from the left, exactly like Python tuples, so a heap of tuples is ordered by `f` first. This is the same trick you would use with `heapq.heappush(pq, (f, -g, r, c))`. Storing `-g` rather than `g` is the tie-break from C1: among equal `f`, the *larger* `g` has the *smaller* `-g` and so comes out first. `top()` and `pop()` are two calls, the same as `front()` and `pop()` on the queue in A2, and `const auto [f, negG, r, c] = open.top();` unpacks the tuple the way `f, neg_g, r, c = heapq.heappop(pq)` does.
- **Push duplicates, skip stale ones.** Suppose (2,2) is already on the list with `g = 6` and we now find a way to reach it with `g = 4`. In a textbook you would "decrease the key" of the existing entry. Neither `std::priority_queue` nor `heapq` can reach into the heap and edit an entry. So we push a *second* entry for (2,2) with the better `f` and leave the old one where it is. The better entry comes out first (smaller `f`), we deal with the cell and mark it `closed`. Later the old entry surfaces; `if (closed[idx]) continue;` throws it away. This is called *lazy deletion*. The heap can hold a few entries per cell instead of one, at most one per incoming edge, which on a grid is 4 or 8. Cheap, and no hand-written heap needed.
- `std::vector<char> closed, not std::vector<bool>.` `vector<bool>` is a special case in C++ that packs eight flags into a byte and behaves strangely (you cannot take a reference to one element). A vector of `char` is a plain byte array. The project uses `std::uint8_t` for the same reason.
- **Rebuilding the path.** `parent[ni] = idx` records, for every cell, which cell we stepped in from. When the goal is taken we walk `parent` links from the goal back to the start (the start's parent is `-1`, which stops the loop), collecting cells goal-first, then `std::reverse` flips the vector in place. Python: `path.reverse()` or `path[::-1]`.
- **-1 as "unset" for g.** Every move costs 1 here, so `g` is an `int` and `-1` is a safe sentinel. In the project moves can cost `sqrt(2)`, so `g` is a `double` and "unset" is `+infinity`, which is cleaner: `new_g < infinity` is always true the first time, so the `g[ni] == -1 ||` part of the check disappears.

Python (heapq)	C++	Note
<code>pq = []; heapq.heappush(pq, (f, -g, r, c))</code>	<code>std::priority_queue<Entry, std::vector<Entry>, std::greater<Entry>> open; open.push({f, -g, r, c});</code>	Must say <code>greater</code> to get smallest-first.
<code>f, ng, r, c = heapq.heappop(pq)</code>	<code>const auto [f, negG, r, c] = open.top(); open.pop();</code>	Two calls.
<code>while pq:</code>	<code>while (!open.empty())</code>	Not truthy in C++.
<code>path[::-1]</code>	<code>std::reverse(path.begin(), path.end());</code>	In place.
nested <code>def h(r, c):</code>	<code>auto h = [&](int r, int c) { ... };</code>	<code>[&]</code> captures surrounding variables.

In your own words: "Compared with BFS: the queue becomes a min-heap on f, I push a fresh entry whenever a cell's g improves and skip stale entries via a closed set, I stop when the goal is popped, and I rebuild the path from a parent array and reverse it. `std::priority_queue` is a max-heap by default, so the comparator is `std::greater`."

C3. 8 directions, diagonal cost, and the right heuristic

Robots do not only move up, down, left and right. Allowing the four diagonal moves as well ("8-connected") gives shorter, more natural paths. Three things change.

A diagonal step costs $\sqrt{2}$. Moving one cell right and one cell down at the same time covers the diagonal of a square with side 1. Pythagoras: $\sqrt{1*1 + 1*1} = \sqrt{2} = 1.414\dots$. If you charged 1 for a diagonal, the planner would treat diagonals as free shortcuts and zig-zag diagonally everywhere. If you charged 2, it would never use one. 1.414 is the truth, and it is what the project charges.

Manhattan is now a bad guess. Say the goal is 2 rows down and 3 columns right, with nothing in the way:

	c0	c1	c2	c3	
r0	S	.	.	.	
r1	.	x	.	.	two diagonals, then one step right
r2	.	.	x	G	cost = 1.414 + 1.414 + 1 = 3.83

Manhattan says $2 + 3 = 5$. The true cost is 3.83. The guess is *bigger* than the truth: it over-estimates, so it is not admissible on an 8-connected grid, and rule 2 from C1 says A* can now return a longer path than necessary. (The project's `MatchesDijkstraOnRandomMaps` test would catch it.)

The right guess is the *octile* distance: you take $\min(dr, dc)$ diagonal steps, which cover one row and one column each, and then $\max(dr, dc) - \min(dr, dc)$ straight steps for what is left over:

```
octile = max(dr, dc) + (sqrt(2) - 1) * min(dr, dc)
        = 3 + 0.414 * 2 = 3.83           for dr = 2, dc = 3
```

That is exactly the true cost when nothing is in the way, so it never over-estimates, and it is as tight as a guess can be. The straight-line (Euclidean) distance $\sqrt{dr*dr + dc*dc}$ is $\sqrt{13} = 3.61$ here: also safe, but looser, so A* takes more cells before it is sure.

Guess	Formula	Safe on 4-connected	Safe on 8-connected
Zero	0	yes (it is Dijkstra)	yes
Manhattan	$dr + dc$	yes, and exact on an open map	no , over-estimates diagonals
Octile	$\max(dr, dc) + (\sqrt{2} - 1) * \min(dr, dc)$	yes, but loose	yes, and exact on an open map
Euclidean	$\sqrt{dr*dr + dc*dc}$	yes, loose	yes, loose

(dr and dc mean $|\text{row} - \text{goal row}|$ and $|\text{col} - \text{goal col}|$.) The rule: use the guess that is exact on an open map for your move set. Manhattan for 4 directions, octile for 8.

Do not cut corners. Look at this robot R wanting to step diagonally to (1,1):

	c0	c1
r0	R	#
r1	.	.

The diagonal move passes right through the corner where the wall at $(0,1)$ touches $(1,0)$. A point could squeeze through; a robot with any width would clip the wall. So the project refuses a diagonal step unless *both* cells it brushes past, $(r + dr, c)$ and $(r, c + dc)$, are free. Here $(0,1)$ is blocked, so the robot goes $(0,0) \rightarrow (1,0) \rightarrow (1,1)$ instead: cost 2, not 1.414. And in this map:

```

      c0  c1
r0    R   #
r1    #   .

```

both brushed cells are walls, and $(1,1)$ is simply unreachable. The NoCornerCutting test checks both cases.

In your own words: "With 8 directions a diagonal costs $\sqrt{2}$. Manhattan then over-estimates and is not admissible; octile is exact on an open grid and is the right heuristic. A diagonal step is only allowed when both orthogonal neighbours are free, otherwise the robot clips the corner."

C4. The project, file by file

The project lives in `code/03-grids-bfs-dfs/astar/`:

astar/	
CMakeLists.txt	how to build the library, the demo and the tests
include/astar/grid.hpp	Grid: the map (flat byte array, inBounds, isFree, fromAscii)
include/astar/astar.hpp	the public interface: Cell, Connectivity, Heuristic, Result, aStar(), dijkstra()
src/astar.cpp	the planner itself
src/demo.cpp	astar_demo: reads a map file, draws the path
tests/astar_test.cpp	13 GoogleTest cases
maps/sample.txt	an ASCII map with S and G
README.md	design notes (the same points as this section, shorter)

Two ideas that are new if you have only ever written single-file C++:

- **Headers and sources.** A `.hpp` file *declares* things (the shape of a struct, the signature of a function). A `.cpp` file *defines* them (the bodies). Anyone who wants to use the planner includes the header; the compiled body lives in a library they link against. Python has no split like this because it has no separate compile step.
- **A library, a demo and a test as three targets.** The planner is compiled once into `libastar.a`. The demo and the test executables each link it. Change the planner, rebuild, and both pick it up.

Read the files in this order.

CMakeLists.txt

CMake is the build description, the closest thing C++ has to a `pyproject.toml` plus a Makefile. It says what to build from which sources, with which flags, and what depends on what. `cmake -S . -B build` reads this file and writes a Makefile into `build/`; `cmake --build build` runs it.

```

cmake_minimum_required(VERSION 3.16)
project(astar_grid LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Release)
endif()

# Library: the planner itself. PUBLIC include dir so consumers get it too.
add_library(astar src/astar.cpp)
target_include_directories(astar PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include)
target_compile_options(astar PRIVATE -Wall -Wextra -Wpedantic)

# Demo: astar_demo [maps/sample.txt] [4|8]

```

```

add_executable(aster_demo src/demo.cpp)
target_link_libraries(aster_demo PRIVATE aster)
target_compile_options(aster_demo PRIVATE -Wall -Wextra -Wpedantic)

# Tests. libgtest-dev on Ubuntu 22.04 ships GTestConfig.cmake, so find_package
# gives us the GTest::gtest_main imported target. Fallback: build from source.
enable_testing()
find_package(GTest QUIET)
if(NOT GTest_FOUND AND EXISTS /usr/src/googletest)
    add_subdirectory(/usr/src/googletest ${CMAKE_BINARY_DIR}/googletest EXCLUDE_FROM_ALL)
    add_library(GTest::gtest_main ALIAS gtest_main)
elseif(NOT GTest_FOUND)
    message(FATAL_ERROR "GoogleTest not found: apt install libgtest-dev")
endif()

add_executable(aster_test tests/aster_test.cpp)
target_link_libraries(aster_test PRIVATE aster GTest::gtest_main)
target_compile_options(aster_test PRIVATE -Wall -Wextra)
add_test(NAME aster_test COMMAND aster_test)

```

What to notice:

- `add_library(aster src/aster.cpp)` makes the planner a library target. `target_include_directories(... PUBLIC include)` says "the headers are in `include/`, and anyone who links `aster` gets that include path too". That is why `demo.cpp` can write `#include "aster/aster.hpp"` without its own setting.
- `add_executable(aster_demo src/demo.cpp)` plus `target_link_libraries(aster_demo PRIVATE aster)` is "build this program, and link the planner into it". Same again for `aster_test`.
- `set(CMAKE_CXX_STANDARD 17)` is the `-std=c++17` you have been typing by hand. `-Wall -Wextra -Wpedantic` are the warnings you have been enabling by hand. The `CMAKE_BUILD_TYPE Release` default turns on `-O3` optimisation.
- `find_package(GTest QUIET)` looks for an installed GoogleTest. On Ubuntu 22.04 `apt install libgtest-dev` provides it and the `GTest::gtest_main` target appears. The `elseif` fallback builds GoogleTest from `/usr/src/googletest` if only the sources are there.
- `enable_testing()` and `add_test(NAME aster_test COMMAND aster_test)` register the test executable with `ctest`, so `ctest --test-dir build` knows what to run.

`include/aster/grid.hpp`

The map. A Grid is a width, a height, and one flat array of bytes, 0 for free and 1 for blocked.

```

// grid.hpp – a binary occupancy grid: 0 free, 1 blocked. Row-major, index r*width+c.
#pragma once

#include <stdint>
#include <string>
#include <vector>

namespace aster {

struct Grid {
    int width = 0;
    int height = 0;
    std::vector<std::uint8_t> cells; // flat, row-major; uint8_t not bool so it is a real byte array

    Grid() = default;
    Grid(int w, int h) : width(w), height(h), cells(static_cast<std::size_t>(w) * h, 0) {}

    int index(int r, int c) const { return r * width + c; }
    bool inBounds(int r, int c) const { return r >= 0 && r < height && c >= 0 && c < width; }
    bool isFree(int r, int c) const { return inBounds(r, c) && cells[index(r, c)] == 0; }
    void setBlocked(int r, int c, bool blocked = true) { cells[index(r, c)] = blocked ? 1 : 0; }

    // '#' is blocked, every other character ('.', 'S', 'G', ' ') is free.
    // Throws std::invalid_argument if rows have different lengths.
    static Grid fromAscii(const std::vector<std::string>& rows);
};

```

```
};

} // namespace astar
```

What to notice:

- `#pragma once` means "if this header is included twice in one file, ignore the second time". Every header has it (or an equivalent `#ifndef` guard).
- `namespace astar { ... }` groups everything under a name, like a Python module. Users write `astar::Grid`. It stops a `Grid` from a different library colliding with ours.
- **A flat `std::vector<std::uint8_t>`**. One allocation of `width * height` bytes, cell `(r, c)` at `r * width + c`. This is the C2 layout and it is the layout of a ROS `OccupancyGrid` and of a `nav2` costmap: a byte array with a width and a height. It is also faster than a vector of vectors, because the whole map is one contiguous block in memory. `uint8_t` rather than `bool` for the `vector<bool>` reason from C2.
- `struct` with functions inside it. `index`, `inBounds`, `isFree`, `setBlocked` are *member functions*, the same as methods on a Python class; `this` is implicit like `self`. The `const` after `inBounds(int r, int c)` `const` promises the function does not change the grid, which is what allows calling it on a `const Grid&`.
- `isFree` does the bounds check *and* the wall check. So the planner can ask "is the neighbour free?" in one call and never index out of range. That is the A1 rule packaged up.
- `Grid(int w, int h) : width(w), height(h), cells(static_cast<std::size_t>(w) * h, 0) {}` is a *constructor* with a *member initialiser list*: the part after the colon sets each field directly. `Grid()` = default; also keeps the do-nothing constructor so an empty `Grid` can be declared and filled later.
- `static Grid fromAscii(...)` is a function attached to the type rather than to an object, like a `@classmethod` factory in Python. Its body is in `astar.cpp`. `'#'` is a wall, every other character is free, so `S` and `G` are free cells. Rows of different lengths throw `std::invalid_argument`, and a test checks that.

`include/astar/astar.hpp`

The public interface of the planner: the types you pass in and the type you get back.

```
// astar.hpp – A* on a Grid. Dijkstra is A* with the zero heuristic.
#pragma once

#include <cstdint>
#include <functional>
#include <vector>

#include "astar/grid.hpp"

namespace astar {

struct Cell {
    int r = 0;
    int c = 0;
};
inline bool operator==(Cell a, Cell b) { return a.r == b.r && a.c == b.c; }
inline bool operator!=(Cell a, Cell b) { return !(a == b); }

enum class Connectivity { Four, Eight };

// Built-in heuristics. All are admissible and consistent for the connectivity
// they are meant for: Manhattan for Four, Octile or Euclidean for Eight.
// Manhattan is NOT admissible on an 8-connected grid (it overestimates diagonals).
enum class Heuristic { Zero, Manhattan, Octile, Euclidean };

using HeuristicFn = std::function<double(Cell, Cell)>;

double zeroHeuristic(Cell a, Cell b);
double manhattan(Cell a, Cell b);
double octile(Cell a, Cell b);
double euclidean(Cell a, Cell b);
HeuristicFn heuristicFor(Heuristic h);
```

```

struct Result {
    bool found = false;
    double cost = 0.0;           // +infinity when !found
    std::vector<Cell> path;      // start..goal inclusive; empty when !found
    std::size_t expanded = 0;   // nodes popped from the open set and closed
};

// Orthogonal moves cost 1, diagonal moves cost sqrt(2). Diagonal moves are
// refused when either orthogonal neighbour is blocked (no corner cutting).
Result aStar(const Grid& grid, Cell start, Cell goal, Connectivity conn, const HeuristicFn& h);
Result aStar(const Grid& grid, Cell start, Cell goal, Connectivity conn, Heuristic h);
Result dijkstra(const Grid& grid, Cell start, Cell goal, Connectivity conn);

} // namespace astar

```

What to notice:

- `struct Cell { int r; int c; }` with `operator==` and `operator!=` defined next to it. That is what lets the tests write `EXPECT_EQ(res.path.front(), start)`: without `==` the compiler has no idea how to compare two `Cells`. In Python you would write `__eq__`.
- `enum class Connectivity { Four, Eight }` is a Python Enum. The `class` keyword makes it *scoped*: you must write `Connectivity::Four`, and it will not silently convert to an `int`. `Heuristic` is the same for the four built-in guesses from C3.
- `using HeuristicFn = std::function<double(Cell, Cell)>;` names a type meaning "anything I can call with two `Cells` that gives back a `double`": a plain function, a lambda, anything. It is the C++ spelling of a `Callable[[Cell, Cell], float]` type hint. It exists so you can pass your *own* guess in; the `CustomHeuristicFunction` test does exactly that with a lambda.
- **Result is what comes back**, and it is a struct rather than a bare number because a planner has four things to say: `found` (is there a path at all), `cost` (the total, in step units, `+infinity` when not found), `path` (every cell from start to goal inclusive, empty when not found), and `expanded` (how many cells were taken off the list). `expanded` is the number you looked at in the C1 pictures: 6 versus 10, 9 versus 25. It is how you *measure* whether a heuristic is helping, and it is what you would log on a robot to tune planning time.
- `aStar` appears twice with the same name and different last parameters. That is *overloading*, which Python does not have: the compiler picks the version by the argument type. One takes any callable, the other takes an enum and looks up the built-in function. `dijkstra` is a third name for `aStar` with the zero guess.
- The `const Grid&` and `const HeuristicFn&` parameters mean "look at the caller's grid, do not copy it". A 1000 by 1000 map is a megabyte; copying it per call would be silly.

src/astar.cpp

The planner. It is the C2 function grown up: double costs, 4 or 8 directions, corner-cutting check, a `Result` instead of an `int`.

```

// astar.cpp – A* with a lazy-deletion binary heap and a closed set.
#include "astar/astar.hpp"

#include <algorithm>
#include <cmath>
#include <cstdlib>
#include <limits>
#include <queue>
#include <stdexcept>

namespace astar {

Grid Grid::fromAscii(const std::vector<std::string>& rows) {
    Grid g(rows.empty() ? 0 : static_cast<int>(rows[0].size()), static_cast<int>(rows.size()));
    for (int r = 0; r < g.height; ++r) {

```

```

        if (static_cast<int>(rows[r].size()) != g.width) throw std::invalid_argument("ragged ascii map");
        for (int c = 0; c < g.width; ++c) g.cells[g.index(r, c)] = rows[r][c] == '#' ? 1 : 0;
    }
    return g;
}

namespace {
constexpr double kSqrt2 = 1.4142135623730951;
// First four entries are orthogonal (cost 1), last four diagonal (cost sqrt2).
constexpr int kDr[8] = {-1, 0, 1, 0, -1, -1, 1, 1};
constexpr int kDc[8] = {0, 1, 0, -1, -1, 1, -1, 1};

struct Node {
    double f;
    double g;
    int idx;
};

// priority_queue is a max-heap, so "greater" puts the smallest f on top.
// Tie-break on larger g: among equal f, prefer the node closer to the goal,
// which avoids sweeping a whole plateau of equal-f cells on open maps.
struct NodeGreater {
    bool operator()(const Node& a, const Node& b) const {
        if (a.f != b.f) return a.f > b.f;
        return a.g < b.g;
    }
};

// namespace

double zeroHeuristic(Cell, Cell) { return 0.0; }
double manhattan(Cell a, Cell b) { return std::abs(a.r - b.r) + std::abs(a.c - b.c); }
double octile(Cell a, Cell b) {
    const int dr = std::abs(a.r - b.r), dc = std::abs(a.c - b.c);
    return (dr + dc) + (kSqrt2 - 2.0) * std::min(dr, dc); // = max + (sqrt2-1)*min
}
double euclidean(Cell a, Cell b) { return std::hypot(a.r - b.r, a.c - b.c); }

HeuristicFn heuristicFor(Heuristic h) {
    switch (h) {
        case Heuristic::Manhattan: return manhattan;
        case Heuristic::Octile: return octile;
        case Heuristic::Euclidean: return euclidean;
        case Heuristic::Zero: break;
    }
    return zeroHeuristic;
}

Result aStar(const Grid& grid, Cell start, Cell goal, Connectivity conn, const HeuristicFn& h) {
    Result res;
    res.cost = std::numeric_limits<double>::infinity();
    if (!grid.isFree(start.r, start.c) || !grid.isFree(goal.r, goal.c)) return res;

    const int n = grid.width * grid.height;
    const double inf = std::numeric_limits<double>::infinity();
    std::vector<double> gScore(n, inf);
    std::vector<int> parent(n, -1);
    std::vector<char> closed(n, 0);
    std::priority_queue<Node, std::vector<Node>, NodeGreater> open;

    const int s = grid.index(start.r, start.c);
    const int t = grid.index(goal.r, goal.c);
    gScore[s] = 0.0;
    open.push({h(start, goal), 0.0, s});
    const int nDirs = conn == Connectivity::Four ? 4 : 8;

    while (!open.empty()) {
        const Node cur = open.top();
        open.pop();
        // Lazy deletion: a node can sit in the heap several times with different
        // g. Only the first (best) pop counts; later copies are stale and skipped.
        if (closed[cur.idx]) continue;
        closed[cur.idx] = 1;
        ++res.expanded;
    }
}

```

```

    if (cur.idx == t) {
        res.found = true;
        res.cost = cur.g;
        for (int i = t; i != -1; i = parent[i]) res.path.push_back({i / grid.width, i % grid.width});
        std::reverse(res.path.begin(), res.path.end());
        return res;
    }

    const int r = cur.idx / grid.width, c = cur.idx % grid.width;
    for (int k = 0; k < nDirs; ++k) {
        const int nr = r + kDr[k], nc = c + kDc[k];
        if (!grid.isFree(nr, nc)) continue;
        // No corner cutting: a diagonal step needs both orthogonal cells free,
        // otherwise a real robot would clip the obstacle corner.
        if (k >= 4 && (!grid.isFree(r + kDr[k], c) || !grid.isFree(r, c + kDc[k]))) continue;
        const int ni = grid.index(nr, nc);
        // With a consistent heuristic a closed node already has its optimal g.
        if (closed[ni]) continue;
        const double ng = cur.g + (k < 4 ? 1.0 : kSqrt2);
        if (ng < gScore[ni]) {
            gScore[ni] = ng;
            parent[ni] = cur.idx;
            open.push({ng + h({nr, nc}, goal), ng, ni});
        }
    }
}
return res; // open set exhausted: no path, cost stays +inf, path stays empty
}

Result aStar(const Grid& grid, Cell start, Cell goal, Connectivity conn, Heuristic h) {
    return aStar(grid, start, goal, conn, heuristicFor(h));
}

Result dijkstra(const Grid& grid, Cell start, Cell goal, Connectivity conn) {
    return aStar(grid, start, goal, conn, zeroHeuristic);
}

} // namespace astar

```

What to notice:

- namespace { ... } with no name is an *anonymous namespace*: everything inside is private to this file, like a leading underscore in Python. `kSqrt2`, the direction arrays, `Node` and `NodeGreater` are implementation details nobody else needs.
- **The direction arrays are ordered on purpose.** Entries 0 to 3 are up, right, down, left; entries 4 to 7 are the diagonals. So `k < 4 ? 1.0 : kSqrt2` gives the step cost, and `nDirs` is 4 or 8 depending on `Connectivity`, and the same loop serves both.
- `struct Node { double f; double g; int idx; }` is the heap entry, a struct instead of the tuple from C2. `NodeGreater` is the comparison, written out by hand: `a.f > b.f` for smallest-*f*-first (the "backwards" comparator again), and on equal *f*, `a.g < b.g` so the larger *g* wins. That is the C1 tie-break; the README calls it out because without it the `HeuristicReducesExpansions` test fails on an open map.
- `gScore` is double with `+infinity` as "unset", as promised in C2. `parent` is int with `-1`, `closed` is a byte per cell.
- The early return: if the start or the goal is a wall or outside the map, return a `Result` with `found = false` and `cost = infinity` before doing anything. `expanded` is 0.
- The loop is the C2 loop. `if (closed[cur.idx]) continue;` is the lazy deletion. `++res.expanded` counts every cell that is really taken. `if (cur.idx == t)` rebuilds the path from `parent` and reverses it.
- **The corner-cutting check** is the line `if (k >= 4 && (!grid.isFree(r + kDr[k], c) || !grid.isFree(r, c + kDc[k]))) continue;`. For a diagonal move (`k >= 4`) it looks at the two cells the robot would brush past and refuses the move if either is blocked. C3 drew this.
- **if (closed[ni]) continue; when looking at neighbours.** This skips a neighbour that is already finished, and it is only correct because the heuristics are *consistent* (C1, rule 2): with a consistent guess, the first time a cell is taken its *g* is already the best possible, so a finished cell can never be improved.

Hand in an inconsistent guess through `HeuristicFn` (for example `2 * manhattan`, "weighted A*") and the planner still terminates, but the path may be a little longer than optimal. That trade is sometimes made on purpose to plan faster.

- The octile body is written as $(dr + dc) + (\sqrt{2} - 2) * \min(dr, dc)$, which is the same as $\max + (\sqrt{2} - 1) * \min$ from C3 after a little algebra ($dr + dc = \max + \min$).
- `heuristicFor` is a switch on the enum that returns the matching function. The case `Heuristic::Zero`: `break;` followed by `return zeroHeuristic;` outside the switch is a style that keeps the compiler happy about every path returning something.
- Cost: every cell is taken at most once and pushes at most 8 entries, each heap operation is $\log$ of the heap size, so the time is proportional to $(\text{cells} \times \text{directions} \times \log \text{ cells})$, written $O(E \log V)$. Memory is three vectors of one entry per cell plus the heap.

tests/astar_test.cpp

The proof. If you have never used GoogleTest, here is all you need:

1. `#include <gtest/gtest.h>`, and link with `GTest::gtest_main`, which supplies `main()` for you.
2. `TEST(SuiteName, TestName) { ... }` defines one test; every `TEST` runs automatically.
3. `EXPECT_EQ(a, b)`, `EXPECT_TRUE(x)`, `EXPECT_FALSE(x)`, `EXPECT_LT(a, b)` check a fact and *carry on* if it fails, so one test can report several failures. `EXPECT_NEAR(a, b, 1e-9)` is the `pytest.approx` for doubles; `EXPECT_DOUBLE_EQ` is the same with a rounding-error tolerance; `EXPECT_THROW(expr, ExceptionType)` is `pytest.raises`.
4. `ASSERT_TRUE(x)` and friends check and *stop this test* on failure. Use them before code that would crash if the fact were false, like `path.front()` on a path that might be empty.
5. Run `./build/astar_test` to see every test, or `./build/astar_test --gtest_filter='AStar.NoPath'` for one.

```
// astar_test.cpp - GoogleTest suite for the grid A*.
#include <cmath>
#include <cstdlib>
#include <limits>
#include <random>
#include <string>
#include <vector>

#include <gtest/gtest.h>

#include "astar/astar.hpp"

using astar::Cell;
using astar::Connectivity;
using astar::Grid;
using astar::Heuristic;
using astar::Result;

namespace {
constexpr double kSqrt2 = 1.4142135623730951;

Grid openGrid(int w, int h) { return Grid(w, h); }

// Every consecutive pair of cells is one king move apart, every cell is free,
// and the summed step costs equal the reported cost.
void expectContiguous(const Grid& g, const Result& res, Cell start, Cell goal) {
    ASSERT_TRUE(res.found);
    ASSERT_FALSE(res.path.empty());
    EXPECT_EQ(res.path.front(), start);
    EXPECT_EQ(res.path.back(), goal);
    double sum = 0.0;
    for (std::size_t i = 0; i < res.path.size(); ++i) {
        EXPECT_TRUE(g.isFree(res.path[i].r, res.path[i].c));
        if (i == 0) continue;
        const int dr = std::abs(res.path[i].r - res.path[i - 1].r);
        const int dc = std::abs(res.path[i].c - res.path[i - 1].c);
        EXPECT_LE(dr, 1);
```

```

    EXPECT_LE(dc, 1);
    EXPECT_FALSE(dr == 0 && dc == 0);
    sum += (dr + dc == 2) ? kSqrt2 : 1.0;
}
EXPECT_NEAR(sum, res.cost, 1e-9);
}
} // namespace

TEST(Grid, FromAsciiAndBounds) {
    const Grid g = Grid::fromAscii({"..#", "S.G"});
    EXPECT_EQ(g.width, 3);
    EXPECT_EQ(g.height, 2);
    EXPECT_TRUE(g.isFree(0, 0));
    EXPECT_FALSE(g.isFree(0, 2));
    EXPECT_TRUE(g.isFree(1, 0)); // 'S' and 'G' are free cells
    EXPECT_FALSE(g.inBounds(-1, 0));
    EXPECT_FALSE(g.inBounds(0, 3));
    EXPECT_FALSE(g.isFree(2, 0)); // out of bounds counts as not free
    EXPECT_THROW(Grid::fromAscii({"...", ".."}), std::invalid_argument);
}

TEST(AStar, StraightLineFourConnected) {
    const Grid g = openGrid(5, 1);
    const Result res = astar::aStar(g, {0, 0}, {0, 4}, Connectivity::Four, Heuristic::Manhattan);
    ASSERT_TRUE(res.found);
    EXPECT_DOUBLE_EQ(res.cost, 4.0);
    EXPECT_EQ(res.path.size(), 5u);
    expectContiguous(g, res, {0, 0}, {0, 4});
}

TEST(AStar, AroundObstacle) {
    // Wall down column 2 with a gap at the bottom row.
    const Grid g = Grid::fromAscii({
        "..#..",
        "..#..",
        "..#..",
        "....."});
    const Result res = astar::aStar(g, {0, 0}, {0, 4}, Connectivity::Four, Heuristic::Manhattan);
    ASSERT_TRUE(res.found);
    EXPECT_DOUBLE_EQ(res.cost, 10.0); // down 3, right 4, up 3
    expectContiguous(g, res, {0, 0}, {0, 4});
    for (const auto& cell : res.path) EXPECT_NE(cell.c == 2 && cell.r < 3, true);
}

TEST(AStar, NoPath) {
    const Grid g = Grid::fromAscii({
        ".#.",
        ".#.",
        ".#."});
    const Result res = astar::aStar(g, {0, 0}, {0, 2}, Connectivity::Eight, Heuristic::Octile);
    EXPECT_FALSE(res.found);
    EXPECT_TRUE(res.path.empty());
    EXPECT_TRUE(std::isinf(res.cost));
    EXPECT_EQ(res.expanded, 3u); // the whole left column was explored before giving up
}

TEST(AStar, BlockedStartOrGoal) {
    const Grid g = Grid::fromAscii({"#.", ".."});
    EXPECT_FALSE(astar::aStar(g, {0, 0}, {1, 1}, Connectivity::Four, Heuristic::Manhattan).found);
    EXPECT_FALSE(astar::aStar(g, {1, 1}, {0, 0}, Connectivity::Four, Heuristic::Manhattan).found);
    EXPECT_FALSE(astar::aStar(g, {1, 1}, {5, 5}, Connectivity::Four, Heuristic::Manhattan).found);
}

TEST(AStar, StartEqualsGoal) {
    const Grid g = openGrid(3, 3);
    const Result res = astar::aStar(g, {1, 1}, {1, 1}, Connectivity::Four, Heuristic::Manhattan);
    ASSERT_TRUE(res.found);
    EXPECT_DOUBLE_EQ(res.cost, 0.0);
    ASSERT_EQ(res.path.size(), 1u);
    EXPECT_EQ(res.path[0], (Cell{1, 1}));
    EXPECT_EQ(res.expanded, 1u);
}

```

```

TEST(AStar, EightConnectedDiagonalCostsSqrt2) {
    const Grid g = openGrid(3, 3);
    const Result res = astar::aStar(g, {0, 0}, {2, 2}, Connectivity::Eight, Heuristic::Octile);
    ASSERT_TRUE(res.found);
    EXPECT_NEAR(res.cost, 2.0 * kSqrt2, 1e-12);
    EXPECT_EQ(res.path.size(), 3u);
    // Four-connected on the same grid must walk the long way.
    const Result four = astar::aStar(g, {0, 0}, {2, 2}, Connectivity::Four, Heuristic::Manhattan);
    EXPECT_DOUBLE_EQ(four.cost, 4.0);
}

TEST(AStar, NoCornerCutting) {
    // One orthogonal neighbour blocked: the diagonal is refused, go around (cost 2, not sqrt2).
    const Grid oneBlocked = Grid::fromAscii({".#", ".."});
    const Result r1 = astar::aStar(oneBlocked, {0, 0}, {1, 1}, Connectivity::Eight, Heuristic::Octile);
    ASSERT_TRUE(r1.found);
    EXPECT_DOUBLE_EQ(r1.cost, 2.0);
    EXPECT_EQ(r1.path.size(), 3u);
    // Both orthogonal neighbours blocked: squeezing between them is impossible.
    const Grid pinched = Grid::fromAscii({".#", "#."});
    const Result r2 = astar::aStar(pinched, {0, 0}, {1, 1}, Connectivity::Eight, Heuristic::Octile);
    EXPECT_FALSE(r2.found);
}

TEST(AStar, PathIsContiguousOnMaze) {
    const Grid g = Grid::fromAscii({
        "S..#.....",
        ".#.#.####.",
        ".#...#....",
        ".####.#.#.",
        ".....#..G"});
    for (Connectivity conn : {Connectivity::Four, Connectivity::Eight}) {
        const Heuristic h = conn == Connectivity::Four ? Heuristic::Manhattan : Heuristic::Octile;
        const Result res = astar::aStar(g, {0, 0}, {4, 9}, conn, h);
        expectContiguous(g, res, {0, 0}, {4, 9});
    }
}

TEST(AStar, MatchesDijkstraOnRandomMaps) {
    std::mt19937 rng(42); // fixed seed: the test is deterministic
    std::bernoulli_distribution blocked(0.3);
    std::uniform_int_distribution<int> coord(0, 19);
    int comparedFound = 0;
    for (int trial = 0; trial < 60; ++trial) {
        Grid g(20, 20);
        for (auto& cell : g.cells) cell = blocked(rng) ? 1 : 0;
        const Cell s{coord(rng), coord(rng)}, t{coord(rng), coord(rng)};
        const Connectivity conn = trial % 2 ? Connectivity::Eight : Connectivity::Four;
        for (Heuristic h : {conn == Connectivity::Four ? Heuristic::Manhattan : Heuristic::Octile,
                           Heuristic::Euclidean}) {
            const Result a = astar::aStar(g, s, t, conn, h);
            const Result d = astar::dijkstra(g, s, t, conn);
            EXPECT_EQ(a.found, d.found);
            if (!a.found) continue;
            ++comparedFound;
            EXPECT_NEAR(a.cost, d.cost, 1e-9);
            expectContiguous(g, a, s, t);
            expectContiguous(g, d, s, t);
        }
    }
    EXPECT_GT(comparedFound, 20); // sanity: the random maps were not all disconnected
}

TEST(AStar, DijkstraIsAStarWithZeroHeuristic) {
    const Grid g = openGrid(15, 15);
    const Result d = astar::dijkstra(g, {7, 7}, {14, 0}, Connectivity::Four);
    const Result z = astar::aStar(g, {7, 7}, {14, 0}, Connectivity::Four, Heuristic::Zero);
    EXPECT_DOUBLE_EQ(d.cost, z.cost);
    EXPECT_EQ(d.expanded, z.expanded);
}

```

```

TEST(AStar, HeuristicReducesExpansions) {
    // Start in the middle so Dijkstra has to grow a full diamond before it
    // touches the corner goal; A* with a good heuristic heads straight there.
    const Grid g = openGrid(41, 41);
    const Cell s{20, 20}, t{40, 40};
    const Result d4 = astar::dijkstra(g, s, t, Connectivity::Four);
    const Result a4 = astar::aStar(g, s, t, Connectivity::Four, Heuristic::Manhattan);
    EXPECT_DOUBLE_EQ(a4.cost, d4.cost);
    EXPECT_LT(a4.expanded * 4, d4.expanded);

    const Result d8 = astar::dijkstra(g, s, t, Connectivity::Eight);
    const Result a8 = astar::aStar(g, s, t, Connectivity::Eight, Heuristic::Octile);
    const Result e8 = astar::aStar(g, s, t, Connectivity::Eight, Heuristic::Euclidean);
    EXPECT_NEAR(a8.cost, d8.cost, 1e-9);
    EXPECT_NEAR(e8.cost, d8.cost, 1e-9);
    EXPECT_LT(a8.expanded * 4, d8.expanded);
    EXPECT_LE(a8.expanded, e8.expanded); // octile is the tighter bound, so it expands no more
}

TEST(AStar, CustomHeuristicFunction) {
    const Grid g = openGrid(6, 6);
    // Weighted A* (w = 2): still finds a path, cost may be suboptimal in general,
    // but on an open grid with Manhattan it stays optimal.
    const astar::HeuristicFn weighted = [](Cell a, Cell b) { return 2.0 * astar::manhattan(a, b); };
    const Result w = astar::aStar(g, {0, 0}, {5, 5}, Connectivity::Four, weighted);
    ASSERT_TRUE(w.found);
    EXPECT_DOUBLE_EQ(w.cost, 10.0);
}

```

What to notice:

- `expectContiguous` is a helper used by several tests. It checks the whole *shape* of an answer, not just the cost: the path starts at `start`, ends at `goal`, every cell is free, every consecutive pair is one move apart (row and column each change by at most 1, and not both 0), and the summed step costs (1 or `sqrt2` each) equal the reported `cost`. Any bug in the parent array or the path reversal fails here.
- `Grid::fromAscii({"S...", ".##.", "...G"})` style maps are used everywhere. The tests are readable as pictures.

What each test proves:

Test	Map	Proves
<code>Grid.FromAsciiAndBounds</code>	<code>..# / S.G</code>	The parser: width, height, # blocked, S/G free, out-of-bounds is not free, ragged rows throw.
<code>AStar.StraightLineFourConnected</code>	5 by 1 open	The simplest possible path: cost 4, 5 cells.
<code>AStar.AroundObstacle</code>	wall with a gap at the bottom	The detour is found and costs 10; no path cell is inside the wall.
<code>AStar.NoPath</code>	full-height wall	<code>found == false</code> , empty path, cost is infinite, and <code>expanded == 3</code> (the whole reachable side was searched before giving up).
<code>AStar.BlockedStartOrGoal</code>	2 by 2 with a wall	A start or goal on a wall, or outside the map, gives <code>found == false</code> without crashing.
<code>AStar.StartEqualsGoal</code>	3 by 3 open	Cost 0, path of one cell, one expansion.
<code>AStar.EightConnectedDiagonalCostsSqrt2</code>	3 by 3 open	Corner to corner is $2 * \sqrt{2}$ with 8 directions and 4 with 4 directions.
<code>AStar.NoCornerCutting</code>	the two C3 pictures	One blocked side forces a cost-2 detour; two blocked sides make the goal unreachable.
<code>AStar.PathIsContiguousOnMaze</code>	a 5 by 10 maze	The path shape is valid for both connectivities.

Test	Map	Proves
AStar.MatchesDijkstraOnRandomMaps	60 random 20 by 20 maps, 30% walls, fixed seed	A with each admissible guess gives the same cost* as Dijkstra, both connectivities. This is the test that catches a non-admissible heuristic, and the strongest one in the file.
AStar.DijkstraIsAStarWithZeroHeuristic	15 by 15 open	dijkstra() and aStar(..., Zero) agree on cost and on expanded.
AStar.HeuristicReducesExpansions	41 by 41 open, start in the middle	A* takes fewer than a quarter of the cells Dijkstra takes, and octile takes no more than Euclidean.
AStar.CustomHeuristicFunction	6 by 6 open	A lambda can be passed as the heuristic (weighted A*, 2 * manhattan).

Read `MatchesDijkstraOnRandomMaps` twice. `std::mt19937 rng(42)` is a random generator with a fixed seed, so the "random" maps are the same every run and a failure can be reproduced. `std::bernoulli_distribution blocked(0.3)` gives `true` 30% of the time. Cross-checking a fast algorithm against a slow one you trust, on many random inputs, is the single most useful testing idea you can take from this chapter.

src/demo.cpp

A small command-line program: read a map file, find S and G, run A* and Dijkstra, print the map with the path drawn as *, and print the cost and both expansion counts.

```
// demo.cpp – astar_demo [map.txt] [4|8]. Reads an ASCII map with 'S' and 'G',
// runs A*, prints the map with the path drawn as '*'.
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

#include "astar/astar.hpp"

int main(int argc, char** argv) {
    const std::string file = argc > 1 ? argv[1] : "maps/sample.txt";
    const bool eight = argc > 2 && std::string(argv[2]) == "8";

    std::ifstream in(file);
    if (!in) {
        std::cerr << "cannot open " << file << "\n";
        return 1;
    }
    std::vector<std::string> rows;
    for (std::string line; std::getline(in, line);)
        if (!line.empty()) rows.push_back(line);

    astar::Cell start{-1, -1}, goal{-1, -1};
    for (int r = 0; r < static_cast<int>(rows.size()); ++r)
        for (int c = 0; c < static_cast<int>(rows[r].size()); ++c) {
            if (rows[r][c] == 'S') start = {r, c};
            if (rows[r][c] == 'G') goal = {r, c};
        }
    if (start.r < 0 || goal.r < 0) {
        std::cerr << "map needs an S and a G\n";
        return 1;
    }

    const astar::Grid grid = astar::Grid::fromAscii(rows);
    const auto conn = eight ? astar::Connectivity::Eight : astar::Connectivity::Four;
    const auto h = eight ? astar::Heuristic::Octile : astar::Heuristic::Manhattan;
    const astar::Result res = astar::aStar(grid, start, goal, conn, h);
    const astar::Result dij = astar::dijkstra(grid, start, goal, conn);
```

```

for (const auto& cell : res.path)
    if (cell != start && cell != goal) rows[cell.r][cell.c] = '*';
for (const auto& row : rows) std::cout << row << "\n";
if (!res.found) {
    std::cout << "no path\n";
    return 0;
}
std::cout << "cost " << res.cost << ", steps " << res.path.size() - 1
    << ", A* expanded " << res.expanded << ", Dijkstra expanded " << dij.expanded << "\n";
return 0;
}

```

What to notice:

- `int main(int argc, char** argv)` is how a C++ program receives command-line arguments: `argc` is how many, `argv[i]` is the *i*-th as a C string, `argv[0]` is the program name. Python's `sys.argv`. `argc > 1 ? argv[1] : "maps/sample.txt"` is "use the first argument if given, else a default".
- `std::ifstream in(file);` opens a file for reading; `if (!in)` is the "could not open" check. `for (std::string line; std::getline(in, line);)` reads it line by line, the same as `for line in open(file)`. Empty lines are skipped.
- The map is read as a `std::vector<std::string>` first so we can find S and G and later overwrite cells with `*` for printing; `Grid::fromAscii` turns the same rows into the byte grid the planner wants.
- It runs both `aStar` and `dijkstra` so the last line shows the two expansion counts side by side. That is exercise 2 below, already built in.

maps/sample.txt

```

S.....#.....
.####.#.#####.
.#...#.#...#..
.#.####.#.#.#..
.#.....#.#.#..
.#####.#.##..
.....#....G

```

A 7 by 16 map. The top-left has a spiral that leads nowhere useful; the way out is down the left edge and along the bottom.

C5. Build it, run it, break it

Build and test. From the repository root:

```

cd "code/03-grids-bfs-dfs/astar"
cmake -S . -B build           # configure: read CMakeLists.txt, write build/Makefile
cmake --build build           # compile the library, the demo and the tests
ctest --test-dir build        # run the registered tests

```

Expected from `ctest`:

```

1/1 Test #1: astar_test ..... Passed    0.00 sec

100% tests passed, 0 tests failed out of 1

```

For the per-test view run the test binary directly:

```
./build/astar_test
```

```

[ RUN      ] Grid.FromAsciiAndBounds
[          OK ] Grid.FromAsciiAndBounds (0 ms)

```

```
[ RUN      ] AStar.StraightLineFourConnected
[         OK ] AStar.StraightLineFourConnected (0 ms)
...
[ RUN      ] AStar.CustomHeuristicFunction
[         OK ] AStar.CustomHeuristicFunction (0 ms)
[ PASSED   ] 13 tests.
```

Run the demo. The second argument is 4 or 8 for the connectivity.

```
./build/astar_demo maps/sample.txt 4
```

```
S.....#.....
*#####.#####.
*#...#.#####.
*#...#.#####.
*#...#.#####.
*#...#.#####.
*#####*#####.
*****#*****G
cost 31, steps 31, A* expanded 50, Dijkstra expanded 66
```

A* took 50 cells off its list, Dijkstra (same code, zero guess) took 66. The map has 71 free cells. The heuristic helps, but on a twisty map with the goal behind walls it helps less than on an open one, because the guess "straight-line distance" is optimistic about every wall.

```
./build/astar_demo maps/sample.txt 8
```

```
S.....#.....
*#####.#####.
*#...#.#####.
*#...#.#####.
*#...#.#####.
*#...#.#####.
*#####*#####.
*****#*****G
cost 31, steps 31, A* expanded 52, Dijkstra expanded 67
```

Same path with 8 directions. Every diagonal that would shorten the route on this map clips a corner, so the corner cutting rule refuses it and the 8-connected answer is identical. The expansion counts differ by a couple of cells because the octile guess gives slightly different f values, so ties are broken in a different order.

Exercise 1: a map with no path. Create maps/nopath.txt:

```
S..#...
...#...
...#...
...#...G
```

```
./build/astar_demo maps/nopath.txt 4
```

```
S..#...
...#...
...#...
...#...G
no path
```

No crash, no -1 in a double, no exception: found is false, cost is +infinity, path is empty. Inside Result, expanded is 12, the number of cells on the start's side of the wall, because the search had to exhaust them all before it could be sure. Callers test found first, always.

Rebuild and run just that test:

```
cmake --build build
./build/astar_test --gtest_filter='*Tiny*
```

```
[ RUN      ] AStar.TinyExampleFromTheTutorial
[ OK      ] AStar.TinyExampleFromTheTutorial (0 ms)
[ PASSED  ] 1 test.
```

The `6u` is an unsigned literal, because `expanded` and `path.size()` are unsigned types and the compiler warns when you compare signed with unsigned (the `A1 static_cast<int>` issue from the other side). Now try breaking things and watch which test catches it: change `a.f > b.f` to `a.f < b.f` in `NodeGreater`, or delete the corner-cutting line, or swap `0ctile` for `Manhattan` in the 8-connected tests. Rebuild, run `ctest`, read the failure message. Put it back.

Where this runs on a real robot. The `nav2` global planner (`NavFn`) is Dijkstra or A over the costmap, with cells near obstacles made expensive by the inflation layer (the 01 Matrix distance transform from A6). The *Smac planners* are A over a lattice of motion primitives instead of grid cells. Both have exactly this structure: a heap of `f`, a closed set, a parent array, a heuristic that must not over-estimate. On a 1000 by 1000 costmap that is a few million heap operations, a few tens of milliseconds, which is why a global plan at 1 Hz is easy.

What to write in your CV about this.

Implemented a grid path planner (A and Dijkstra, 4- and 8-connected, admissible heuristics, no corner cutting) in C++17 as a CMake library with a demo tool and a 13-case GoogleTest suite, including randomised cross-checks of A against Dijkstra.

In your own words: "The planner is a library with a `Grid` of flat bytes and an `aStar` function returning found, cost, path and expansion count. The open set is a `priority_queue` with lazy deletion, the closed set a byte per cell, ties go to larger `g`, diagonals cost $\sqrt{2}$ and never cut corners. The tests cross-check A* against Dijkstra on seeded random maps, and the demo prints both expansion counts so I can see the heuristic working."

Part D — Check yourself

Answer these in your own words before marking the phase done in the tracker. Every answer is in this chapter.

1. In `grid[r][c]`, which index goes down and which goes right? What is `grid.size()` and `grid[0].size()`?
2. Write the `dr/dc` arrays for 4 directions from memory. Now for 8.
3. Why must the bounds check come before `grid[nr][nc]`? What happens in Python versus C++ if you get that wrong?
4. Describe BFS as a picture. Why does it find the shortest path?
5. Why do you mark a cell visited when you push it rather than when you pop it?
6. How do you count "rings" (minutes, steps) in BFS? Why must you snapshot the queue size?
7. What changes in BFS when there are several starting points? What does each cell's distance then mean?
8. Describe DFS as a picture. Name two questions DFS answers and one it does not.
9. What does the `&` in `std::vector<std::vector<bool>>& visited` do, and what breaks without it?
10. When can recursive DFS crash, and what do you replace it with?
11. What is the one-line difference between DFS and backtracking? Why is backtracking slower?
12. For "number of ways to reach each cell moving only right or down", what is the transition and what are the base cases?

13. Which problem in this chapter is the wavefront planner? Which is the distance transform?
14. In A*, what are g, h and f? What happens when h is zero? What happens when h guesses too high?
15. Why does `std::priority_queue` need a "backwards" comparator to pop the smallest f, and why do we push duplicates instead of updating entries?